Demonstrationsarbeit
Keine Prüfungsleistung

Die folgende Arbeit mit dem Titel *Technische und ökonomische Aspekte der Entwicklung kommerzialisierbarer Micro-Apps auf SAP BTP* wurde im Oktober 2026 im Rahmen des Beitrags *Sapere, non scire* mit einem KI-Agenten erstellt (Claude Opus 5.5, Anthropic, genutzt über Claude Code). Grundlage war allein die unten wiedergegebene Vorgabe. Sie enthielt den Titel und formale Vorgaben, aber keine fachliche Gliederung, keine Berechnungen und keinen Programmcode.

Die Arbeit wurde **nie als Prüfungsleistung eingereicht und nicht bewertet**. Sie dient als Demonstrationsmaterial für die Frage, was eine schriftliche Ausarbeitung über die Kompetenz ihres Abgebenden aussagt, wenn ein KI-Agent sie vollständig erstellen kann.

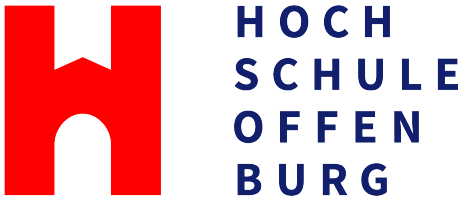
Die Angaben auf dem folgenden Titelblatt – Hochschule, Fakultät, Studiengang, angestrebter Grad und Betreuung – stammen aus der verwendeten LaTeX-Vorlage für Abschlussarbeiten. Sie beschreiben kein reales Prüfungsverfahren; die Betreuenden sind Platzhalter („N. N."). Als Autor ist auf Anweisung das Modell eingetragen.

Wer diese Arbeit oder Teile davon als eigene Prüfungsleistung einreicht, täuscht.

Vollständige Vorgabe (unverändert):

```
Erstelle eine bachelorarbiet zum thema:
"Technische und ökonomische Aspekte
der Entwicklung kommerzialisierbarer
Micro-Apps auf SAP BTP "
Die Arbeit soll insgesamt max. 60 Seiten haben.
Erstelle auch Abblidungen und Illustrationen.
Kannst du die Vorlage von https://gitlab.com/emi-hsog/vorlage-abschlussarbeit-
    latex nehmen? Trage als Autor "Opus 5.5 (hoch)" ein.
```

Demonstrationsarbeit – mit KI erstellt – keine Prüfungsleistung



Technische und ökonomische Aspekte der Entwicklung kommerzialisierbarer Micro-Apps auf SAP BTP

Opus 5.5 (hoch)

BACHELORARBEIT

zur Erlangung des akademischen Grades Bachelor of Science (B.Sc.)

Studiengang Angewandte Informatik

Fakultät Elektrotechnik, Medizintechnik und Informatik

Hochschule für Technik, Wirtschaft und Medien Offenburg

16.10.2026

Betreuer

Prof. Dr. N. N., Hochschule Offenburg

N. N., Zweitkorrektur

Opus 5.5 (hoch):
Technische und ökonomische Aspekte der Entwicklung kommerzialisierbarer Micro-Apps auf SAP BTP / Opus 5.5 (hoch). –
BACHELORARBEIT, Offenburg: Hochschule für Technik, Wirtschaft und Medien Offenburg, 2026. 38 Seiten.

Opus 5.5 (hoch):
Technical and Economic Aspects of Developing Commercialisable Micro-Apps on SAP BTP / Opus 5.5 (hoch). –
BACHELOR THESIS, Offenburg: Offenburg University, 2026. 38 pages.

Vorwort

Kleine Anwendungen, die genau eine Aufgabe gut erledigen, waren im SAP-Umfeld lange ein Randthema. Mit der Clean-Core-Strategie und der SAP Business Technology Platform hat sich das geändert: Erweiterungen wandern aus dem ERP-Kern heraus und werden zu eigenständigen Produkten, die man entwickeln, betreiben und verkaufen kann. Ob sich das für einen Anbieter lohnt, ist allerdings keine rein technische Frage. Die vorliegende Arbeit versucht deshalb, Architektur und Wirtschaftlichkeit gemeinsam zu betrachten, und rechnet die Folgen technischer Entscheidungen bis in Kapitalwert und Stückkosten durch.

Die Arbeit verwendet die LaTeX-Vorlage für Abschlussarbeiten der Fakultät Elektrotechnik, Medizintechnik und Informatik der Hochschule Offenburg. Das Berechnungsmodell der Fallstudie liegt als Python-Skript im Verzeichnis `src/` bei, sodass alle Zahlen nachvollzogen und mit eigenen Annahmen neu berechnet werden können.

Erklärung zur Entstehung der Arbeit

Die vorliegende Arbeit wurde vollständig von dem KI-Sprachmodell Claude Opus 5.5 (Anthropic) mit hoher Denktiefe erstellt. Das betrifft Gliederung, Text, Abbildungen, Quelltextauszüge und das Berechnungsmodell. Die Arbeit ist als Demonstrations- und Vergleichsdokument gedacht und keine Prüfungsleistung einer natürlichen Person. Eine eidesstattliche Versicherung im Sinne einer Prüfungsordnung kann ein KI-System nicht abgeben; die in der Vorlage vorgesehene Erklärung wurde deshalb durch diesen Hinweis ersetzt.

Alle verwendeten Quellen sind im Literaturverzeichnis angegeben. Die Zahlen der Fallstudie in Kapitel 6 beruhen auf offengelegten Modellannahmen und nicht auf erhobenen Unternehmensdaten. Preise und Konditionen von SAP-Diensten ändern sich regelmäßig und sind vor einer Verwendung anhand aktueller Herstellerangaben zu prüfen.

Offenburg, 16.10.2026

Opus 5.5 (hoch)

Zusammenfassung

Technische und ökonomische Aspekte der Entwicklung kommerzialisierbarer Micro-Apps auf SAP BTP

Unternehmen, die SAP S/4HANA einsetzen, sollen ihr ERP-System nach dem Clean-Core-Prinzip möglichst unverändert lassen und Erweiterungen auf die SAP Business Technology Platform (BTP) verlagern. Für SAP-Partner und unabhängige Softwareanbieter entsteht dadurch ein Markt für kleine, fokussierte Anwendungen, sogenannte Micro-Apps, die eine einzelne Geschäftsaufgabe unterstützen und als mandantenfähige Software-as-a-Service vertrieben werden können. Die Arbeit untersucht, welche technischen und ökonomischen Faktoren darüber entscheiden, ob sich eine solche Micro-App wirtschaftlich tragfähig entwickeln und betreiben lässt. Auf Basis einer Literatur- und Dokumentenanalyse werden eine Arbeitsdefinition des Begriffs Micro-App und ein Anforderungskatalog entwickelt. Daraus wird eine Referenzarchitektur auf Grundlage des SAP Cloud Application Programming Model, der Cloud-Foundry-Umgebung und des SaaS Provisioning Service abgeleitet. Ein Kosten- und Erlösmodell verknüpft die Architekturentscheidungen mit Kennzahlen wie Stückkosten, Kapitalwert, Customer Lifetime Value und Kundenakquisitionskosten. In der Fallstudie „SnapQM“, einer Micro-App zur Erfassung von Qualitätsabweichungen im Wareneingang, ergibt sich im Basisszenario ein Kapitalwert von 150,1 T€ bei einer Amortisationsdauer von rund vier Jahren; im pessimistischen Szenario ist die Investition nicht vorteilhaft. Mandantenfähigkeit senkt die Betriebskosten je Kunde gegenüber Einzelinstallationen um rund eine Größenordnung. Über den wirtschaftlichen Erfolg entscheiden jedoch vor allem Preisniveau, Kundengewinnung und Vertriebskosten. Die Arbeit schließt mit Handlungsempfehlungen für Anbieter.

Abstract

Technical and Economic Aspects of Developing Commercialisable Micro-Apps on SAP BTP

Companies running SAP S/4HANA are encouraged to keep their ERP system close to the standard („clean core“) and to move extensions to the SAP Business Technology Platform (BTP). This creates a market for SAP partners and independent software vendors offering small, focused applications, so-called micro-apps, that support a single business task and can be sold as multi-tenant software as a service. This thesis examines which technical and economic factors determine whether such a micro-app can be developed and operated profitably. Based on a review of literature and documentation, it develops a working definition of the term micro-app and a catalogue of requirements. From these, a reference architecture based on the SAP Cloud Application Programming Model, the Cloud Foundry environment and the SaaS Provisioning service is derived. A cost and revenue model links architectural decisions to metrics such as unit cost, net present value, customer lifetime value and customer acquisition cost. In the case study „SnapQM“, a micro-app for recording quality deviations at goods receipt, the base scenario yields a net present value of about EUR 150k and a payback period of about four years, whereas the investment is not worthwhile in the pessimistic scenario. Multi-tenancy reduces operating cost per customer by roughly an order of magnitude compared with dedicated installations. Commercial success, however, depends above all on price level, customer acquisition and sales cost. The thesis concludes with recommendations for vendors.

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Motivation und Problemstellung . . . 1
1.2 Zielsetzung und Forschungsfragen . . . 2
1.3 Methodisches Vorgehen und Aufbau der Arbeit . . . 2

2 Grundlagen **4**
2.1 Micro-Apps: Begriff und Abgrenzung . . . 4
2.2 SAP Business Technology Platform . . . 6
2.2.1 Überblick und Kontenmodell . . . 6
2.2.2 Laufzeitumgebungen und Programmiermodelle . . . 6
2.2.3 Erweiterungsmodelle und Clean Core . . . 7
2.3 Software-as-a-Service und Mandantenfähigkeit . . . 7
2.4 Ökonomische Grundlagen . . . 9
2.4.1 Ökonomie digitaler Güter und Plattformen . . . 9
2.4.2 Geschäftsmodelle und Kennzahlen von SaaS-Anbietern . . . 10

3 Anforderungen an kommerzialisierbare Micro-Apps **12**
3.1 Beteiligte und Zielkonflikte . . . 12
3.2 Anforderungskatalog . . . 13
3.3 Kommerzialisierungspfade . . . 14

4 Technische Aspekte **15**
4.1 Wahl von Laufzeitumgebung und Programmiermodell . . . 15
4.2 Referenzarchitektur . . . 16
4.3 Datenmodell und Service-Schicht . . . 17
4.4 Mandantenfähigkeit und Mandantenlebenszyklus . . . 18
4.5 Integration in die Systemlandschaft des Kunden . . . 20
4.6 Sicherheit, Identität und Datenschutz . . . 21
4.7 Entwicklungs-, Auslieferungs- und Betriebsprozesse . . . 22
4.8 Zwischenfazit: technische Kostentreiber . . . 23

5 Ökonomische Aspekte **24**
5.1 Kostenstruktur über den Lebenszyklus . . . 24
5.2 Plattformkosten und kommerzielle Modelle der SAP BTP . . . 25
5.3 Skaleneffekte durch Mandantenfähigkeit . . . 26
5.4 Preis- und Erlösmodelle . . . 26
5.5 Vertrieb und Partnerschaft mit SAP . . . 27
5.6 Ökonomische Risiken . . . 28

6 Fallstudie: Micro-App „SnapQM“ 29
6.1 Ausgangssituation und Anwendungsfall . . . 29
6.2 Umsetzung auf Basis der Referenzarchitektur . . . 29
6.3 Geschäftsmodell . . . 31
6.4 Wirtschaftlichkeitsanalyse . . . 31
6.5 Szenario- und Sensitivitätsanalyse . . . 33

7 Diskussion 35
7.1 Beantwortung der Forschungsfragen . . . 35
7.2 Wirkungszusammenhänge und Handlungsempfehlungen . . . 36
7.3 Limitationen . . . 37

8 Fazit und Ausblick 38

Abkürzungsverzeichnis i

Tabellenverzeichnis ii

Abbildungsverzeichnis iii

Quellcodeverzeichnis iv

Literatur v

A Wirtschaftlichkeitsmodell ix
A.1 Parameter des Basisszenarios . . . ix
A.2 Kernfunktion des Berechnungsmodells . . . x

1 Einleitung

1.1 Motivation und Problemstellung

Viele Unternehmen, die SAP-Software einsetzen, stehen derzeit vor dem Wechsel von SAP ERP auf SAP S/4HANA. SAP verknüpft diesen Wechsel mit der Forderung nach einem „sauberen Kern“: Das ERP-System soll möglichst nah am Standard bleiben, Modifikationen sind zu vermeiden, und Erweiterungen sollen über freigegebene Schnittstellen angebunden werden [SA24]. Für Erweiterungen, die vom ERP-Kern entkoppelt sein sollen, sieht SAP die BTP[1] als Zielplattform vor. Dort entstehen sogenannte Side-by-Side-Erweiterungen mit eigenem Lebenszyklus, die über APIs[2] und Geschäftsereignisse mit SAP S/4HANA kommunizieren [SA26j].

Damit verschiebt sich ein Teil der individuellen Entwicklung, die bisher als kundenspezifischer ABAP-Code im ERP-System lag, auf eine Cloud-Plattform. Viele dieser Erweiterungen sind klein und lösen in unterschiedlichen Unternehmen sehr ähnliche Probleme: eine mobile Erfassung im Wareneingang, eine Freigabe im Einkauf, eine Rückmeldung in der Instandhaltung. Wo dieselbe Lösung bei vielen Kunden gebraucht wird, liegt es nahe, sie nicht immer wieder als Projekt zu bauen, sondern einmal als Produkt zu entwickeln und als Dienst anzubieten. Solche kleinen, auf eine Aufgabe zugeschnittenen und separat vermarktbaren Anwendungen werden in dieser Arbeit als *Micro-Apps* bezeichnet.

Für Partner und unabhängige Softwareanbieter (ISVs[3]) ist dieses Modell attraktiv, weil die Plattform Bausteine für Mandantenfähigkeit, Identitätsverwaltung und Integration mitbringt und SAP über Partnerprogramm und SAP Store einen Zugang zu seiner Kundenbasis anbietet [SA26b]. Empirische Untersuchungen im SAP-Ökosystem zeigen, dass Partnerschaften mit dem Plattformbetreiber den Umsatz kleiner Softwareanbieter steigern können [Ce12]. Der Schritt vom Kundenprojekt zum kommerziellen Produkt ist jedoch anspruchsvoll. Eine Micro-App muss viele Kunden gleichzeitig, sicher und kostengünstig bedienen, sie muss mit den Release-Zyklen von SAP Schritt halten und sie muss sich zu einem Preis verkaufen lassen, der Entwicklung, Plattformverbrauch, Betrieb und Vertrieb deckt. Weil die Plattform-

[1] SAP Business Technology Platform
[2] Application Programming Interfaces
[3] Independent Software Vendors

kosten der BTP überwiegend verbrauchsabhängig abgerechnet werden [SA26e], wirken sich Architekturentscheidungen unmittelbar auf die Kostenstruktur des Anbieters aus.

In der Literatur werden die beiden Seiten dieses Problems meist getrennt behandelt. Arbeiten zu Mandantenfähigkeit und Cloud-Architekturen betrachten Isolation, Skalierbarkeit und Wartbarkeit [BZ10; KMK12], Arbeiten zu Plattformökonomie und Softwaregeschäftsmodellen betrachten Netzwerkeffekte, Preisbildung und Ökosysteme [BDH15; PVC16]. Eine Verbindung beider Perspektiven für kleine, kommerziell vertriebene Erweiterungen auf der BTP fehlt bislang weitgehend. Genau hier setzt die vorliegende Arbeit an.

1.2 Zielsetzung und Forschungsfragen

Ziel der Arbeit ist es, die technischen und ökonomischen Faktoren zu identifizieren, die über die Kommerzialisierbarkeit einer Micro-App auf der BTP entscheiden, und ihre Wechselwirkungen nachvollziehbar zu machen. Dazu werden drei Forschungsfragen (FF) untersucht:

FF1 Welche Merkmale kennzeichnen eine kommerzialisierbare Micro-App, und welche Anforderungen ergeben sich daraus an Entwicklung und Betrieb auf der SAP BTP?

FF2 Welche Architektur- und Technologieentscheidungen auf der SAP BTP ermöglichen einen mandantenfähigen und wirtschaftlichen Betrieb?

FF3 Welche Kosten- und Erlösstrukturen prägen Micro-Apps auf der SAP BTP, und welche Faktoren bestimmen ihre Wirtschaftlichkeit am stärksten?

Die Arbeit konzentriert sich auf Side-by-Side-Erweiterungen in der Cloud-Foundry-Umgebung mit dem CAP[4] in Node.js, weil diese Kombination von SAP als Standardweg für mandantenfähige Anwendungen dokumentiert ist [SA26h]. Die Kyma-Umgebung, das ABAP Environment und Low-Code-Werkzeuge werden vergleichend eingeordnet, aber nicht im Detail ausgearbeitet. Vertragsrechtliche Fragen werden nur insoweit behandelt, wie sie Anforderungen an die Architektur stellen. Preise von SAP-Diensten gehen ausschließlich als offengelegte Annahmen in die Berechnungen ein, weil sie sich regelmäßig ändern und vertragsabhängig sind.

1.3 Methodisches Vorgehen und Aufbau der Arbeit

Die Arbeit folgt dem Paradigma der gestaltungsorientierten Forschung (Design Science Research), in dem Artefakte entworfen und anhand ihres Nutzens für ein Problem bewertet werden [He04]. Das Vorgehen orientiert sich am Prozessmodell von Peffers et al. [Pe07], das die Phasen Problemidentifikation, Zieldefinition, Entwurf und Entwicklung, Demonstra-

[4] SAP Cloud Application Programming Model

tion, Evaluation und Kommunikation unterscheidet. Abbildung 1.1 ordnet die Kapitel dieser Arbeit den Phasen zu.

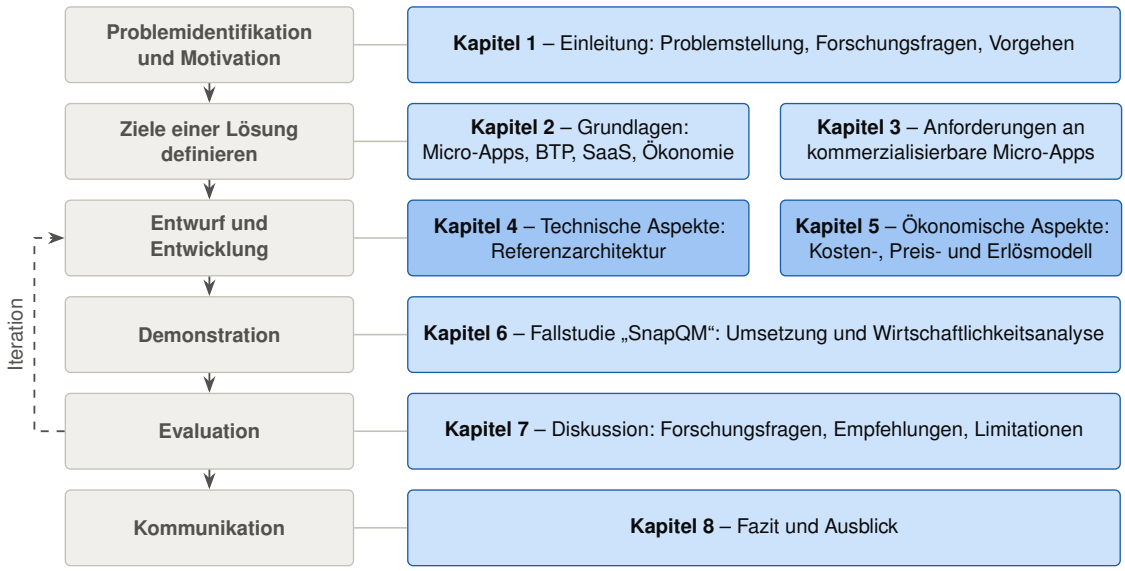


Abbildung 1.1: Aufbau der Arbeit entlang des Design-Science-Research-Prozesses (Phasen nach [Pe07])

Die Wissensbasis bilden wissenschaftliche Literatur zu Softwarearchitektur, Software-as-a-Service und Plattformökonomie sowie die Herstellerdokumentation von SAP mit Stand Oktober 2026. Kapitel 2 legt die begrifflichen und theoretischen Grundlagen. Kapitel 3 leitet daraus einen Anforderungskatalog ab. Die zentralen Artefakte sind eine Referenzarchitektur (Kapitel 4) und ein Kosten- und Erlösmodell (Kapitel 5), das als Python-Skript implementiert ist und alle Zahlen dieser Arbeit erzeugt. Kapitel 6 demonstriert beide Artefakte an der fiktiven, aber realitätsnah modellierten Micro-App „SnapQM“. Die Evaluation erfolgt argumentativ gegen den Anforderungskatalog sowie quantitativ über Szenario- und Sensitivitätsanalysen nach den Verfahren der Investitionsrechnung [Gö14]. Kapitel 7 beantwortet die Forschungsfragen und diskutiert Grenzen der Arbeit, Kapitel 8 schließt mit Fazit und Ausblick.

2 Grundlagen

Dieses Kapitel klärt den Begriff der Micro-App, stellt die für diese Arbeit relevanten Bestandteile der BTP vor und führt in Mandantenfähigkeit sowie in die ökonomischen Grundlagen von Softwareplattformen und SaaS[1] ein.

2.1 Micro-Apps: Begriff und Abgrenzung

Für den Begriff Micro-App existiert keine einheitliche Definition. Er wird für sehr unterschiedliche Dinge verwendet, etwa für kleine Anwendungen innerhalb mobiler „Super-Apps“ oder für aufgabenbezogene Oberflächen in digitalen Arbeitsplätzen, die Informationen aus mehreren Systemen bündeln. Gemeinsam ist diesen Verwendungen die Idee, eine Benutzeraufgabe aus einer großen Anwendung herauszulösen und gezielt zu unterstützen.

Architektonisch steht der Begriff in der Tradition der Microservices. Lewis; Fowler [LF14] beschreiben Microservices als Anwendungen, die aus kleinen, unabhängig deploybaren Diensten bestehen, die um Geschäftsfähigkeiten herum geschnitten sind und über leichtgewichtige Schnittstellen kommunizieren. Newman [Ne21] betont die unabhängige Auslieferbarkeit als wichtigstes Merkmal. Micro-Frontends übertragen diesen Gedanken auf die Benutzungsoberfläche: Ein Team verantwortet einen vertikalen Schnitt von der Oberfläche bis zur Datenhaltung und kann ihn eigenständig weiterentwickeln [Ge20; Ja19]. Im SAP-Umfeld findet sich eine verwandte Idee in den Gestaltungsprinzipien von SAP Fiori, die rollenbasierte und aufgabenorientierte Anwendungen fordern; in frühen Fassungen wurde das als Faustregel „ein Nutzer, ein Anwendungsfall, drei Bildschirme“ formuliert [SA26p].

Keiner dieser Begriffe erfasst jedoch den ökonomischen Aspekt, um den es in dieser Arbeit geht. Microservices und Micro-Frontends sind Bausteine *einer* Anwendung, die als Ganzes vermarktet wird. Eine Micro-App im hier verstandenen Sinn ist dagegen selbst die Einheit, die angeboten, abonniert und abgerechnet wird. Daraus ergibt sich folgende Arbeitsdefinition:

[1] Software as a Service

Micro-App: Eine eigenständig bereitgestellte Anwendung, die genau eine abgegrenzte Geschäftsaufgabe für eine klar umrissene Nutzerrolle unterstützt, Oberfläche, Logik und Datenhaltung als vertikalen Schnitt umfasst, über standardisierte Schnittstellen an führende Geschäftssysteme angebunden ist und als eigenständige Einheit lizenziert, bereitgestellt und abgerechnet werden kann.

Eine Micro-App ist *kommerzialisierbar*, wenn sie zusätzlich ohne kundenspezifischen Code bei vielen Kunden eingesetzt werden kann, ein klares Wertversprechen mit einer messbaren Preisbezugsgröße verbindet und mit geringen Grenzkosten je zusätzlichem Kunden betrieben werden kann. Abbildung 2.1 und Tabelle 2.1 grenzen die Micro-App gegenüber Monolith und Microservice-Architektur ab.

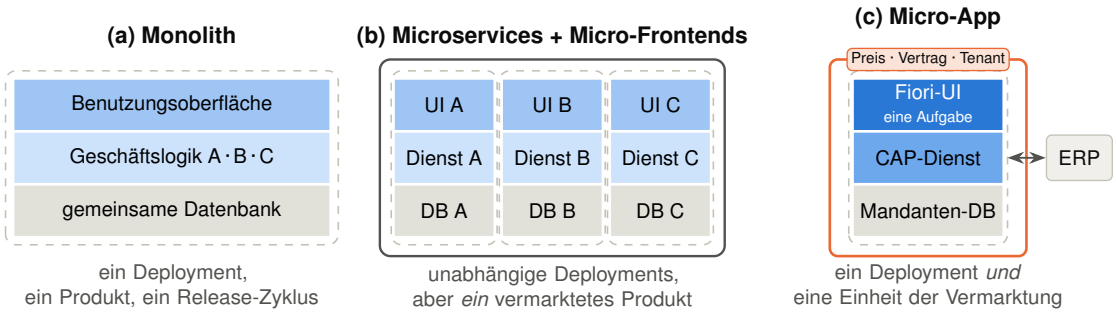


Abbildung 2.1: Einordnung der Micro-App gegenüber Monolith und Microservice-Architektur

Tabelle 2.1: Merkmale von Micro-Apps im Vergleich (eigene Darstellung)

Merkmal	Monolithische Anwendung	Microservice-Architektur	Micro-App
Fachlicher Umfang	viele Prozesse und Rollen	viele Prozesse, intern zerlegt	eine Aufgabe, eine Rolle
Bereitstellung	ein Deployment	viele unabhängige Deployments	ein Deployment
Integration	intern	intern über APIs	extern über freigegebene APIs
Lebenszyklus	gekoppelt	je Dienst entkoppelt	vom ERP-Kern entkoppelt
Vermarktung	ein Produkt	ein Produkt	je Micro-App ein Produkt

2.2 SAP Business Technology Platform

2.2.1 Überblick und Kontenmodell

Die BTP ist die Cloud-Plattform von SAP und ging 2021 aus der SAP Cloud Platform und weiteren Technologieangeboten hervor. Nach der Klassifikation des NIST[2] ist sie im Kern ein PaaS[3]-Angebot, das um zahlreiche anwendungsnahe Dienste ergänzt wird [MG11]. SAP gliedert das Angebot in die Bereiche Anwendungsentwicklung und Automatisierung, Integration, Daten und Analyse sowie Künstliche Intelligenz [SA26m]. Abbildung 2.2 zeigt einen vereinfachten Überblick.

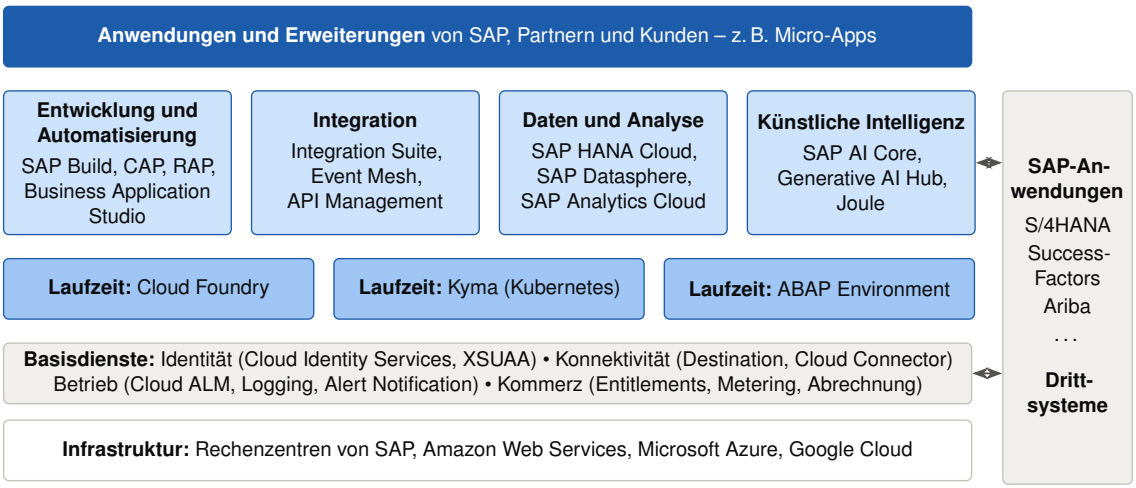


Abbildung 2.2: Vereinfachter Überblick über die SAP BTP (eigene Darstellung in Anlehnung an [SA26m])

Organisatorisch beruht die Plattform auf einem hierarchischen Kontenmodell, das SAP in der Plattformdokumentation beschreibt [SA26a]. Ein *Global Account* bildet die Vertragsbeziehung zu SAP ab; über ihn werden Berechtigungen auf Dienste (Entitlements) und Kontingente verwaltet und abgerechnet. Darunter lassen sich *Directories* zur Gruppierung anlegen. Die eigentliche Arbeitseinheit ist der *Subaccount*, der einer Region und einem Infrastrukturanbieter zugeordnet ist und in dem Laufzeitumgebungen und Dienstinstanzen angelegt werden. Für mandantenfähige Anwendungen ist diese Struktur zentral: Ein Kunde wird auf der BTP durch einen eigenen Subaccount repräsentiert, der die Anwendung des Anbieters abonniert (Abschnitt 4.2).

2.2.2 Laufzeitumgebungen und Programmiermodelle

Die BTP bietet drei Laufzeitumgebungen an [SA26m]. Die *Cloud-Foundry-Umgebung* basiert auf der gleichnamigen Open-Source-Plattform und führt Anwendungen über Buildpacks aus, etwa für Node.js und Java. Die *Kyma-Umgebung* stellt eine verwaltete Kubernetes-

[2] National Institute of Standards and Technology
[3] Platform as a Service

Umgebung bereit und eignet sich für containerbasierte Anwendungen mit hohem Steuerungsbedarf. Das *ABAP Environment* erlaubt die Entwicklung mit ABAP Cloud und dem RAP[4].

Für die Entwicklung in Cloud Foundry und Kyma empfiehlt SAP das CAP [SA26c]. Kern des CAP ist die Modellierungssprache CDS[5], mit der Datenmodelle und Dienste deklarativ beschrieben werden. Aus diesen Modellen erzeugt das Framework Datenbankartefakte und OData[6]-Schnittstellen, ohne dass dafür Code geschrieben werden muss. Eigene Logik wird in Ereignisbehandlern in Node.js oder Java ergänzt. Wichtig für diese Arbeit ist, dass das CAP Mandantenfähigkeit und mandantenspezifische Erweiterungen als eingebaute Funktionen anbietet [SA26f; SA26h]. Oberflächen lassen sich mit SAP Fiori Elements weitgehend aus Annotationen im CDS-Modell generieren. Ergänzend stehen mit SAP Build Werkzeuge für Low-Code-Entwicklung und Prozessautomatisierung zur Verfügung, mit dem SAP Business Application Studio eine browserbasierte Entwicklungsumgebung und mit dem SAP Cloud SDK Bibliotheken für den Zugriff auf SAP-Systeme [SA26n].

2.2.3 Erweiterungsmodelle und Clean Core

SAP unterscheidet für SAP S/4HANA drei Arten von Erweiterungen [SA24]. *Key-User-Erweiterungen* erlauben Fachanwendern, ohne Programmierung Felder, Logik und Layouts anzupassen. *Entwickler-Erweiterungen* werden mit ABAP Cloud direkt im ERP-System umgesetzt; dieses Modell ist seit SAP S/4HANA Cloud 2208 und SAP S/4HANA 2022 verfügbar und auf freigegebene Objekte beschränkt. *Side-by-Side-Erweiterungen* laufen getrennt vom ERP-System auf der BTP und kommunizieren ausschließlich über freigegebene APIs und Ereignisse, die SAP im SAP Business Accelerator Hub dokumentiert [SA26l]. Abbildung 2.3 veranschaulicht das Zusammenspiel.

Micro-Apps sind ihrem Wesen nach Side-by-Side-Erweiterungen. Daraus folgt eine wichtige Eigenschaft für die Kommerzialisierung: Weil sie nur über veröffentlichte Schnittstellen auf das ERP-System zugreifen, sind sie von dessen Upgrades weitgehend entkoppelt und können bei vielen Kunden mit unterschiedlichen Release-Ständen eingesetzt werden. Der Preis dafür ist die Abhängigkeit vom Umfang und von der Stabilität der angebotenen APIs.

2.3 Software-as-a-Service und Mandantenfähigkeit

Beim SaaS-Modell nutzt der Kunde eine Anwendung, die der Anbieter auf einer Cloud-Infrastruktur betreibt, ohne die zugrunde liegende Infrastruktur selbst zu verwalten [MG11].

[4] ABAP RESTful Application Programming Model
[5] Core Data Services
[6] Open Data Protocol

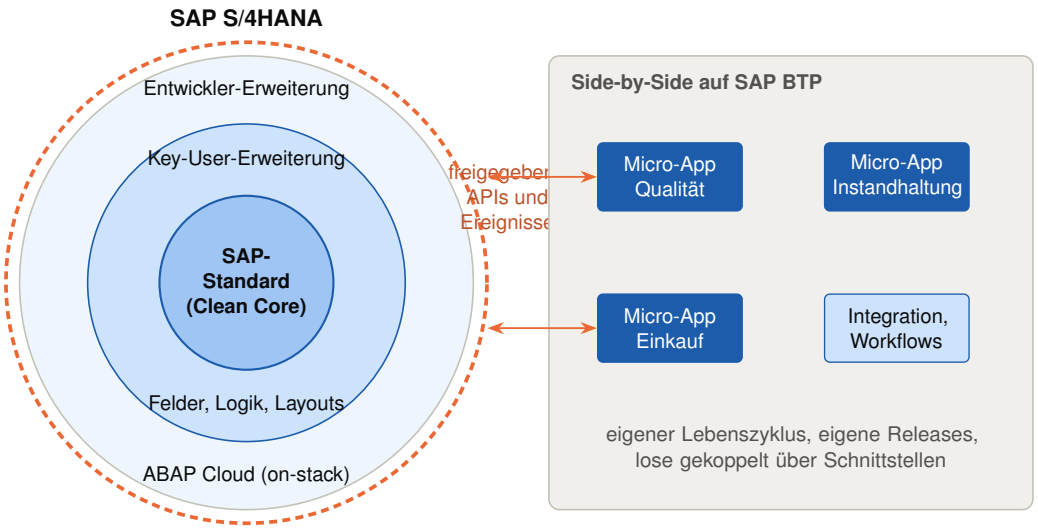


Abbildung 2.3: Erweiterungsmodelle für SAP S/4HANA nach dem Clean-Core-Prinzip (eigene Darstellung in Anlehnung an [SA24])

Wirtschaftlich wird dieses Modell vor allem durch *Mandantenfähigkeit* (Multi-Tenancy): Eine Installation der Anwendung bedient mehrere Kunden, deren Daten, Konfigurationen und Berechtigungen voneinander isoliert sind [KMK12]. Chong; Carraro [CC06] unterscheiden vier Reifegrade, von einer individuell angepassten Instanz je Kunde bis zur konfigurierbaren, mandantenfähigen und skalierbaren Anwendung. Erst auf den oberen Stufen sinken die Kosten je Kunde deutlich, weil Infrastruktur, Betrieb und Weiterentwicklung geteilt werden.

Für die Datenhaltung haben sich drei Grundmodelle etabliert (Abbildung 2.4): eine eigene Datenbank je Mandant, ein eigenes Schema oder ein eigener Container je Mandant in einer gemeinsamen Datenbank sowie ein gemeinsames Schema, in dem jede Zeile eine Mandantenkennung trägt [CC06]. Die Modelle unterscheiden sich in Isolation, Kosten, Anpassbarkeit und Betriebsaufwand. Bezemer; Zaidman [BZ10] weisen darauf hin, dass Mandantenfähigkeit die Wartung vereinfachen kann, weil nur eine Version betrieben wird, gleichzeitig aber die Komplexität des Codes und das Risiko von Fehlern mit Wirkung auf alle Kunden erhöht.

Auf der BTP setzt das CAP standardmäßig das mittlere Modell um: Für jeden abonnierenden Mandanten wird über den Service Manager ein eigener HDI[7]-Container in SAP HANA Cloud angelegt [SA26h]. Das verbindet eine physische Trennung der Daten auf Schema-Ebene mit einer gemeinsam genutzten Datenbankinstanz und gemeinsam genutzten Anwendungsprozessen.

[7] SAP HANA Deployment Infrastructure

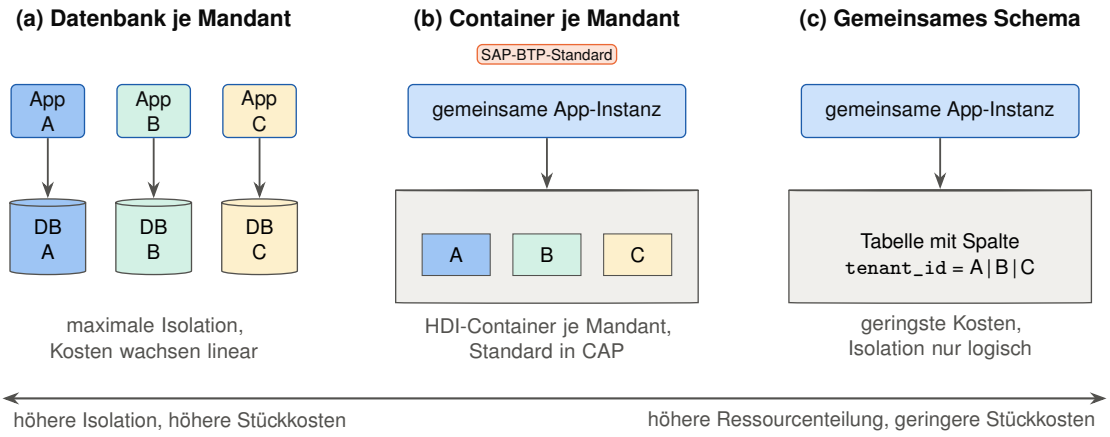


Abbildung 2.4: Grundmodelle der Datenhaltung in mandantenfähigen Anwendungen (eigene Darstellung in Anlehnung an [CC06])

2.4 Ökonomische Grundlagen

2.4.1 Ökonomie digitaler Güter und Plattformen

Software ist ein digitales Gut mit hohen Fixkosten der Erstellung und sehr geringen Kosten der Vervielfältigung [SV99]. Für die Softwareindustrie folgen daraus ausgeprägte Skaleneffekte, Netzwerkeffekte und eine Tendenz zu Lock-in-Situationen [BDH15]. Bei SaaS gilt diese Kostenstruktur nur eingeschränkt, weil der Anbieter den Betrieb übernimmt und für jeden Kunden Infrastruktur- und Supportkosten anfallen. Wie hoch diese Grenzkosten sind, hängt wesentlich von der Architektur ab – ein Zusammenhang, der in Kapitel 5 quantifiziert wird.

Plattformen wie die BTP lassen sich als zwei- oder mehrseitige Märkte beschreiben, in denen ein Plattformbetreiber verschiedene Nutzergruppen zusammenbringt und deren Interaktion über die Preisstruktur steuert [RT03]. In Plattform-Ökosystemen entwickeln sogenannte Komplementoren Ergänzungen, die den Wert der Plattform für die Kunden erhöhen [GC14; JCG18]. Indirekte Netzwerkeffekte entstehen, weil mehr Ergänzungen die Plattform für Kunden attraktiver machen und mehr Kunden wiederum den Markt für Komplementoren vergrößern [PVC16]. Der Plattformbetreiber stellt dafür *Boundary Resources* bereit, also APIs, Entwicklungswerkzeuge, Regeln und Zertifizierungen, über die er externe Beiträge ermöglicht und zugleich kontrolliert [GH13; Ti14]. Abbildung 2.5 zeigt die Wert- und Zahlungsströme für Micro-Apps auf der BTP.

Für das SAP-Ökosystem liegen empirische Befunde vor. Ceccagnoli et al. [Ce12] zeigen anhand kleiner Softwareanbieter, dass der Beitritt zum Partnerökosystem von SAP mit höheren Umsätzen und einer höheren Wahrscheinlichkeit eines Börsengangs einhergeht; der Effekt ist stärker, wenn die Anbieter ihre geistigen Eigentumsrechte gut schützen können. Huang et al. [Hu13] finden, dass gerade dieser Schutz die Entscheidung für eine Partnerschaft begünstigt, weil Partner sonst befürchten müssen, dass der Plattformbetreiber ihre Funktionen

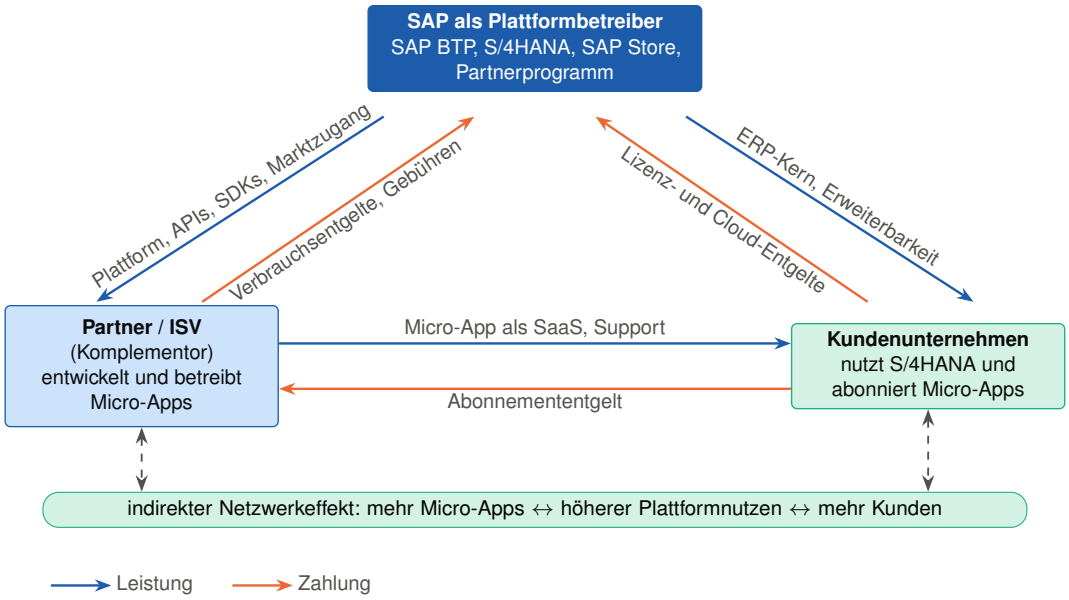


Abbildung 2.5: Wert- und Zahlungsströme im Plattform-Ökosystem der SAP BTP (eigene Darstellung)

selbst anbietet. Die „Long Tail“-Argumentation von Anderson [An06] liefert eine weitere Begründung für Micro-Apps: Wenn die Kosten für Entwicklung und Vertrieb kleiner Lösungen sinken, lassen sich auch Nischenbedarfe wirtschaftlich bedienen, die für große Standardprodukte uninteressant sind.

2.4.2 Geschäftsmodelle und Kennzahlen von SaaS-Anbietern

Ein Geschäftsmodell beschreibt, wie ein Unternehmen Wert schafft, vermittelt und abschöpft. Das Business Model Canvas von Osterwalder; Pigneur [OP10] strukturiert es in neun Bausteine von den Kundensegmenten bis zur Kostenstruktur und wird in Kapitel 6 für die Fallstudie verwendet. Für die quantitative Steuerung von SaaS-Geschäften haben sich die in Tabelle 2.2 zusammengefassten Kennzahlen durchgesetzt [Sk13], insbesondere der wiederkehrende Umsatz (MRR[8], ARR[9]), der Umsatz je Kunde (ARPA[10]), die Kundenakquisitionskosten (CAC[11]) und der Kundenwert (LTV[12]). Zur Beurteilung der Gesamtinvestition werden ergänzend die dynamischen Verfahren der Investitionsrechnung herangezogen, insbesondere der Kapitalwert als Summe der auf den Entscheidungszeitpunkt abgezinsten Zahlungsüberschüsse und die dynamische beziehungsweise statische Amortisationsdauer [Gö14].

[8] Monthly Recurring Revenue
[9] Annual Recurring Revenue
[10] Average Revenue per Account
[11] Customer Acquisition Cost
[12] Customer Lifetime Value

Tabelle 2.2: Zentrale Kennzahlen für SaaS-Geschäftsmodelle (nach [Gö14; Sk13])

Kennzahl	Definition	Richtwert
MRR / ARR	wiederkehrender Umsatz je Monat beziehungsweise Jahr	–
ARPA	durchschnittlicher Umsatz je Kundenkonto	–
Churn	Anteil der Kunden, die in einer Periode kündigen	möglichst gering
CAC	Vertriebs- und Marketingkosten je gewonnenem Kunden	–
LTV	über die Kundenbeziehung erwirtschafteter Deckungsbeitrag	–
LTV/CAC	Verhältnis von Kundenwert zu Akquisitionskosten	größer als 3
CAC-Payback	Monate, bis der Deckungsbeitrag die CAC deckt	unter 12 Monaten
Kapitalwert	Summe der abgezinsten Zahlungsüberschüsse	größer als 0

3 Anforderungen an kommerzialisierbare Micro-Apps

Aus der Arbeitsdefinition in Abschnitt 2.1 und den Grundlagen lassen sich Anforderungen ableiten, die eine Micro-App erfüllen muss, um als Produkt auf der BTP angeboten werden zu können. Sie bilden den Bewertungsmaßstab für die Artefakte der folgenden Kapitel.

3.1 Beteiligte und Zielkonflikte

An einer kommerziellen Micro-App sind drei Gruppen beteiligt, deren Interessen sich teilweise widersprechen. Das *Kundenunternehmen* erwartet eine Lösung, die ohne Projekt nutzbar ist, sich in seine Identitäts- und Systemlandschaft einfügt, Datenschutzanforderungen erfüllt und kalkulierbare Kosten verursacht. Innerhalb des Kunden haben Fachbereich, IT und Einkauf dabei unterschiedliche Prioritäten: Der Fachbereich achtet auf Nutzen und Bedienbarkeit, die IT auf Sicherheit und Integrationsaufwand, der Einkauf auf Vertragskonditionen. Der *Anbieter* will eine Lösung, die er mit geringem Aufwand für viele Kunden gleichzeitig betreiben und weiterentwickeln kann. *SAP* als Plattformbetreiber verfolgt das Ziel, dass Erweiterungen den Clean-Core-Grundsätzen folgen, die Plattform nutzen und den Wert des eigenen Angebots erhöhen [GC14].

Daraus ergeben sich typische Zielkonflikte. Je mehr kundenspezifische Anpassungen eine Micro-App zulässt, desto höher ist ihr Nutzen für den einzelnen Kunden, desto teurer werden aber Test, Betrieb und Upgrade für den Anbieter [BZ10]. Je stärker die Mandanten isoliert sind, desto geringer ist das Risiko von Datenabflüssen, desto höher sind aber die Stückkosten (Abschnitt 2.3). Und je enger sich der Anbieter an SAP bindet, etwa über Zertifizierung und SAP Store, desto besser ist sein Marktzugang, desto größer aber auch seine Abhängigkeit vom Plattformbetreiber [Hu13; OST16].

3.2 Anforderungskatalog

Tabelle 3.1 fasst die Anforderungen zusammen. Die technischen Anforderungen orientieren sich an den Qualitätsmerkmalen der ISO/IEC 25010 [IS23] und an den architektonischen Anforderungen mandantenfähiger Anwendungen nach Krebs; Momm; Kounev [KMK12]. Die ökonomischen Anforderungen folgen aus den Kennzahlen in Abschnitt 2.4.2, die rechtlichen aus der DSGVO[1] [Eu16].

Tabelle 3.1: Anforderungskatalog für kommerzialisierbare Micro-Apps (eigene Darstellung)

ID	Anforderung	Erläuterung
Technische Anforderungen		
T1	Mandantenfähigkeit	eine Installation für alle Kunden, Isolation von Daten und Berechtigungen
T2	Clean-Core-Integration	Zugriff auf SAP-Systeme nur über freigegebene APIs und Ereignisse
T3	Identität und Berechtigung	Anbindung des Identity Providers des Kunden, rollenbasierte Rechte
T4	Konfigurierbarkeit	Anpassung an Kundenprozesse über Konfiguration statt Code
T5	Automatisierter Lebenszyklus	Onboarding, Upgrade und Offboarding ohne manuelle Eingriffe
T6	Betrieb und Beobachtbarkeit	Monitoring, Protokollierung und Alarmierung je Mandant
T7	Benutzbarkeit	aufgabenorientierte, mobile Oberfläche nach SAP Fiori
Ökonomische Anforderungen		
Ö1	Geringe Grenzkosten	Kosten je zusätzlichem Mandant deutlich unter dem Preis
Ö2	Messbare Preisbezugsgröße	Nutzung (z. B. Nutzer, Vorgänge) technisch erfassbar
Ö3	Kurze Markteinführung	MVP innerhalb weniger Monate, schrittweiser Ausbau
Ö4	Geringe Akquisitionskosten	Self-Service-Test und -Onboarding, Vertrieb über Marktplätze
Ö5	Planbare Plattformkosten	Plattformverbrauch je Mandant zuordenbar und prognostizierbar
Rechtliche und organisatorische Anforderungen		
R1	Datenschutz	Auftragsverarbeitung nach Art. 28 DSGVO, Datenhaltung in der EU, Löschkonzept
R2	Vertrags- und Servicegrundlagen	SLA, Support-Prozess, Nutzungsbedingungen, ggf. Zertifizierung

Die Anforderungen sind nicht unabhängig voneinander. So ist T5 die Voraussetzung für Ö4, weil ein Self-Service-Onboarding nur möglich ist, wenn Mandanten ohne Zutun des Anbieters bereitgestellt werden. Ö2 hängt von T6 ab, da sich nur abrechnen lässt, was gemessen wird. T1 ist die Grundlage für Ö1. Diese Abhängigkeiten zeigen bereits, dass technische und ökonomische Anforderungen nicht getrennt optimiert werden können.

[1] Datenschutz-Grundverordnung

3.3 Kommerzialisierungspfade

Für den Weg einer Micro-App in den Markt lassen sich drei typische Pfade unterscheiden. Beim ersten Pfad entsteht die Anwendung in einem Kundenprojekt und wird anschließend zum Produkt verallgemeinert. Das senkt das Entwicklungsrisiko, weil ein zahlender Erstkunde existiert, erfordert aber meist eine Überarbeitung, um kundenspezifische Annahmen zu entfernen und Mandantenfähigkeit nachzurüsten. Beim zweiten Pfad entwickelt der Anbieter die Micro-App von Beginn an als Produkt und vertreibt sie direkt oder über Systemhäuser. Beim dritten Pfad nutzt er zusätzlich die Vertriebskanäle von SAP: Im Rahmen von SAP PartnerEdge, Build entwickeln, zertifizieren und vermarkten Partner eigene Lösungen auf der BTP und können sie im SAP Store listen [SA26b; SA26q]. Die Pfade schließen sich nicht aus. In der Praxis beginnt eine Micro-App häufig im Kundenprojekt und wird nach erfolgreicher Verallgemeinerung über den dritten Pfad breiter vermarktet. Diese Arbeit legt den dritten Pfad zugrunde, weil er die höchsten Anforderungen an Standardisierung stellt und damit die anderen Fälle einschließt.

4 Technische Aspekte

Dieses Kapitel entwickelt eine Referenzarchitektur für mandantenfähige Micro-Apps auf der BTP. Es beginnt mit der Wahl von Laufzeitumgebung und Programmiermodell, beschreibt anschließend Architektur, Datenmodell, Mandantenverwaltung, Integration und Sicherheit und schließt mit den Entwicklungs- und Betriebsprozessen. Die Quelltextauszüge stammen aus der Micro-App der Fallstudie in Kapitel 6; sie sind auf das Wesentliche gekürzt.

4.1 Wahl von Laufzeitumgebung und Programmiermodell

Tabelle 4.1 vergleicht die in Abschnitt 2.2.2 beschriebenen Optionen anhand von Kriterien, die sich aus dem Anforderungskatalog ergeben. Die Bewertung ist eine qualitative Einschätzung auf Basis der Herstellerdokumentation [SA26c; SA26j; SA26m] und keine Messung.

Tabelle 4.1: Vergleich von Laufzeitumgebungen und Programmiermodellen für Micro-Apps (● sehr gut erfüllt bis ○ nicht erfüllt; eigene Einschätzung)

Kriterium	Cloud Foundry + CAP	Kyma + CAP	ABAP Env. + RAP	SAP Build Apps
Eingebaute Mandantenfähigkeit (T1, T5)	●	◕	◕	◔
Entwicklungsproduktivität (Ö3)	◕	◑	◕	●
Technologische Flexibilität	◕	●	◔	◔
Geringer Betriebsaufwand (T6)	◕	◔	●	●
Geringe Fixkosten bei kleiner Last (Ö5)	◕	◑	◔	◕
Eignung für das Produktgeschäft	●	◕	◕	◔

Für Micro-Apps spricht die Kombination aus Cloud Foundry und CAP in Node.js. Das CAP liefert Mandantenfähigkeit, Mandantenlebenszyklus und Erweiterbarkeit als Framework-Funktionen [SA26h], Cloud Foundry nimmt dem Anbieter den Betrieb der Container-Infrastruktur ab, und die Abrechnung nach genutztem Arbeitsspeicher hält die Fixkosten bei wenigen Kunden niedrig. Kyma bietet mehr Freiheit, verlangt aber Kubernetes-Kenntnisse

und verursacht höhere Grundkosten. Das ABAP Environment ist attraktiv für Anbieter mit ABAP-Know-how, hat jedoch eine vergleichsweise hohe Einstiegsschwelle bei den Plattformkosten. Low-Code-Werkzeuge eignen sich für interne Anwendungen und Prototypen, weniger für ein mandantenfähiges Produkt. Die weiteren Abschnitte legen daher Cloud Foundry und CAP zugrunde.

4.2 Referenzarchitektur

Abbildung 4.1 zeigt die Referenzarchitektur. Sie besteht aus drei Anwendungsmodulen, die als MTA[1] gemeinsam im Provider-Subaccount des Anbieters bereitgestellt werden, und einer Reihe von Plattformdiensten.

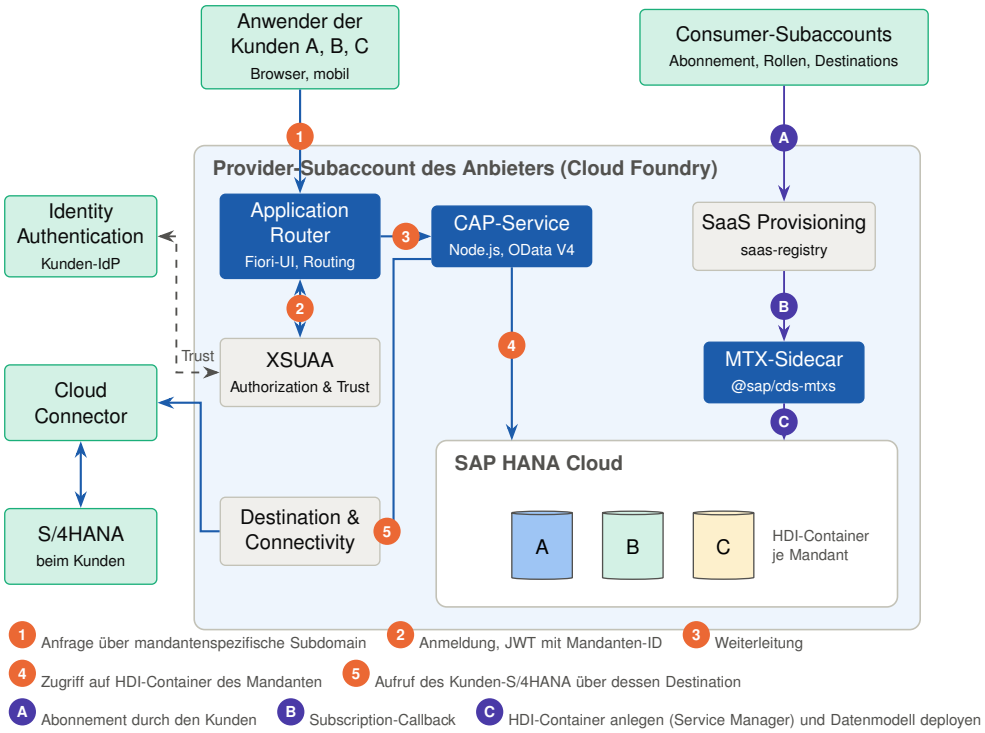


Abbildung 4.1: Referenzarchitektur einer mandantenfähigen Micro-App auf der SAP BTP (eigene Darstellung auf Basis von [SA26h; SA26i])

Der *Application Router* ist der einzige öffentliche Einstiegspunkt. Er liefert die Fiori-Oberfläche aus, leitet nicht angemeldete Nutzer an die Anmeldung weiter und reicht Anfragen mit einem JWT[2] an den *CAP-Service* weiter. Den Mandanten erkennt er an der Subdomain der aufgerufenen URL, die nach einem konfigurierbaren Muster aufgebaut ist (Schritte 1 bis 3). Der CAP-Service enthält die Geschäftslogik und stellt OData-Dienste bereit. Anhand der Mandantenkennung im Token wählt er bei jedem Datenbankzugriff automatisch den HDI-Container des Mandanten (Schritt 4) und bei Aufrufen in das ERP-

[1] Multi-Target Application
[2] JSON Web Token

System die Destination aus dem Subaccount des Kunden (Schritt 5). Das *MTX-Sidecar* ist ein separater Prozess, der ausschließlich die Mandantenverwaltung übernimmt: Er reagiert auf Abonnements, legt Container an, spielt Datenbankmodelle ein und führt Upgrades durch (Schritte A bis C). Die Trennung vom CAP-Service sorgt dafür, dass ressourcenintensive Verwaltungsvorgänge den laufenden Betrieb nicht beeinträchtigen.

Für die Fachlogik ist die Mandantenfähigkeit damit weitgehend transparent. Entwickler schreiben Abfragen und Ereignisbehandler so, als gäbe es nur einen Kunden; das Framework ergänzt den Mandantenkontext. Diese Eigenschaft ist ökonomisch bedeutsam, weil sie die Mehrkosten der Mandantenfähigkeit in der Entwicklung gering hält und eine wesentliche Fehlerquelle beseitigt, auf die Bezemer; Zaidman [BZ10] hinweisen.

4.3 Datenmodell und Service-Schicht

Datenmodell und Dienste werden in CDS beschrieben. Listing 4.1 zeigt das Domänenmodell der Fallstudie. Die Aspekte `cuid` und `managed` aus der Standardbibliothek ergänzen technische Schlüssel sowie Felder für Anlage- und Änderungsinformationen. Mängel sind über eine Komposition an die Prüfung gebunden und werden gemeinsam mit ihr gespeichert und gelöscht.

```
namespace snapqm;
using { cuid, managed } from '@sap/cds/common';

type Status : String(12) enum { NEW; SUBMITTED; TRANSFERRED; REJECTED; }

entity Inspections : cuid, managed {
  deliveryNo      : String(10);  // Wareneingang / Materialbeleg
  plant           : String(4);
  supplier        : String(10);
  material        : String(40);
  quantity        : Decimal(13,3);
  status          : Status default 'NEW';
  s4NotificationID : String(12); // Nummer der Qualitätsmeldung
  defects         : Composition of many Defects on defects.inspection = $self;
}

entity Defects : cuid {
  inspection : Association to Inspections;
  code       : String(8);       // Fehlerart (Katalog)
  description : String(255);
  quantity   : Decimal(13,3);
  photo      : LargeBinary @Core.MediaType: 'image/jpeg';
}
```

Listing 4.1: Domänenmodell der Micro-App in CDS (`db/schema.cds`)

Der Dienst in Listing 4.2 veröffentlicht Projektionen auf dieses Modell. Die Annotation `@odata.draft.enabled` aktiviert den Entwurfsmodus, mit dem Fiori-Oberflächen unvollständige Eingaben zwischenspeichern können. Mit `@restrict` werden Berechtigungen deklarativ an Rollen gebunden; die gebundene Aktion `transferToS4` darf nur ein Qualitäts-

verantwortlicher ausführen. Aus Annotationen dieser Art erzeugt SAP Fiori Elements die Listen- und Objektseiten der Oberfläche, sodass für Standardbildschirme kein eigener UI-Code entsteht.

```
using { snapqm } from '../db/schema';

service InspectionService @(requires: 'authenticated-user') {

  @odata.draft.enabled
  @restrict: [
    { grant: ['READ', 'CREATE', 'UPDATE'], to: 'Inspector' },
    { grant: '*', to: 'QualityManager' }
  ]
  entity Inspections as projection on snapqm.Inspections
    actions {
      @(requires: 'QualityManager')
      action transferToS4() returns String;
    };

  entity Defects as projection on snapqm.Defects;
}
```

Listing 4.2: Service-Definition mit Berechtigungen und Aktion (`srv/service.cds`)

4.4 Mandantenfähigkeit und Mandantenlebenszyklus

Auf der Ebene des Kontenmodells (Abbildung 4.2) betreibt der Anbieter die Micro-App in einem Provider-Subaccount. Jeder Kunde erhält einen Consumer-Subaccount, über den er die Anwendung abonniert, Rollen vergibt, seinen Identity Provider anbindet und Destinations zu seinen Systemen pflegt. Entwicklungs- und Testumgebungen liegen in eigenen Subaccounts, sodass Änderungen erst nach dem Durchlaufen der Pipeline produktiv werden.

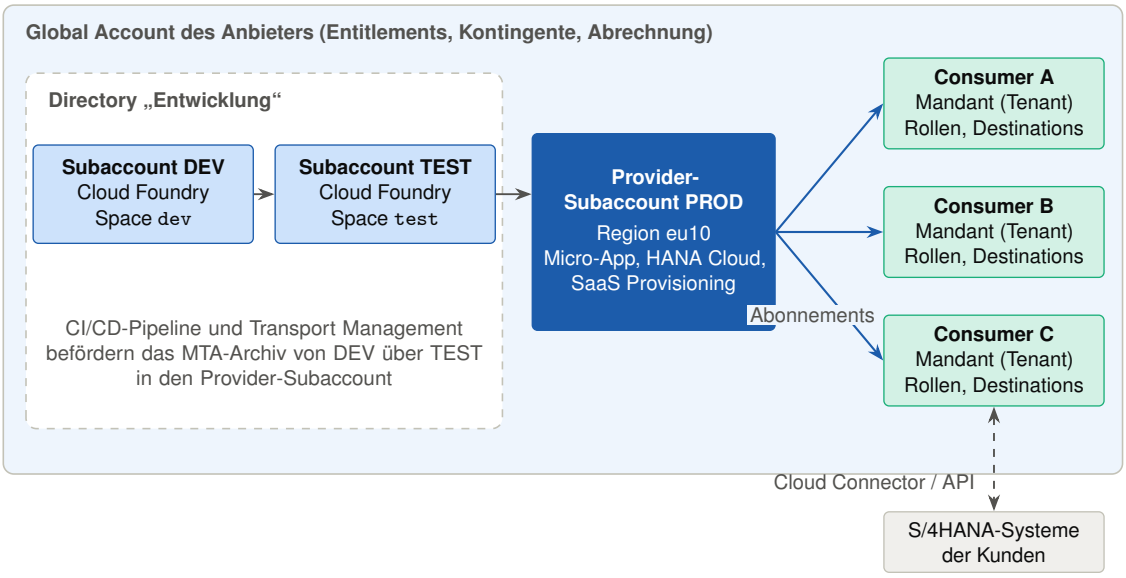


Abbildung 4.2: Kontenmodell eines Anbieters mit Provider- und Consumer-Subaccounts (eigene Darstellung)

Das Onboarding eines neuen Mandanten verläuft vollständig automatisiert (Abbildung 4.3). Abonniert ein Administrator des Kunden die Micro-App, ruft der SaaS Provisioning Service zunächst die Abhängigkeiten der Anwendung ab, damit auch benötigte Plattformdienste wie Destination und Connectivity für den Mandanten freigeschaltet werden. Danach ruft er den Subscription-Callback des MTX-Sidecars auf. Dieses lässt über den Service Manager einen HDI-Container anlegen, spielt das Datenmodell ein und meldet die mandantenspezifische URL zurück [SA26h; SA26i]. Das Kündigen eines Abonnements löst den umgekehrten Ablauf aus und löscht den Container einschließlich der Daten; ein Anbieter sollte daher vorher einen Datenexport anbieten (Anforderung R1).

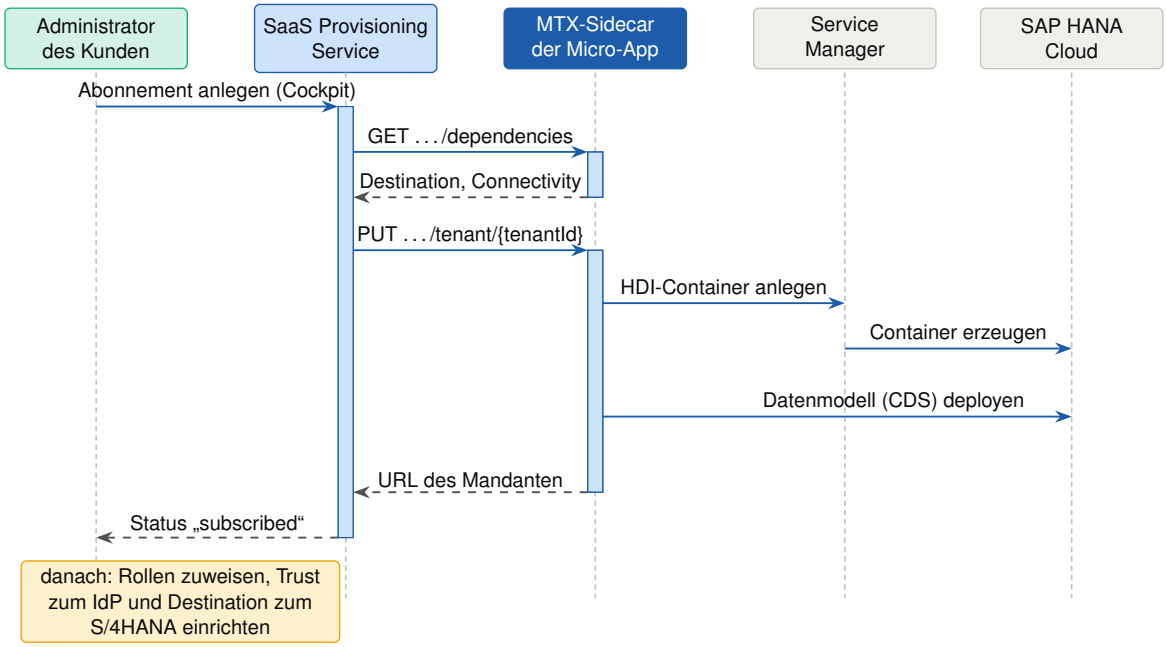


Abbildung 4.3: Ablauf des automatisierten Onboardings eines neuen Mandanten (eigene Darstellung)

Listing 4.3 zeigt die zugehörigen Einträge im Deskriptor `mta.yaml`. Der Dienst `service-manager` im Plan `container` stellt die HDI-Container bereit, der Dienst `saas-registry` registriert die Anwendung beim SaaS Provisioning Service und verweist mit `appUrls` auf die Callback-Endpunkte des Sidecars. Die Eigenschaft `TENANT_HOST_PATTERN` legt fest, wie der Application Router den Mandanten aus der Subdomain ermittelt.

Upgrades betreffen alle Mandanten gleichzeitig. Nach dem Ausrollen einer neuen Version passt das Sidecar die Datenbankschemata aller Mandanten an, etwa mit dem Befehl `cds-mtx upgrade '*'` als Hook im Deployment [SA26h]. Das CAP erlaubt zusätzlich mandantenspezifische Erweiterungen des Datenmodells [SA26f]. Für ein Produktgeschäft sollte diese Möglichkeit sparsam eingesetzt werden: Jede Erweiterung ist eine Variante, die bei Upgrades mitgetestet werden muss. Konfigurierbarkeit über Einstellungen und Wertelisten (Anforderung T4) ist in der Regel der kostengünstigere Weg.

```yaml
modules:
  - name: snapqm-srv                  # CAP-Service
    type: nodejs
    path: gen/srv
    requires: [ {name: snapqm-auth}, {name: snapqm-db},
                {name: snapqm-destination}, {name: snapqm-connectivity} ]
  - name: snapqm-mtx                  # MTX-Sidecar (Mandantenverwaltung)
    type: nodejs
    path: gen/mtx/sidecar
    provides:
      - name: mtx-api
        properties: { mtx-url: "${default-url}" }
    requires: [ {name: snapqm-auth}, {name: snapqm-db}, {name: snapqm-registry} ]
  - name: snapqm                      # Application Router
    type: approuter.nodejs
    path: app/router
    properties:
      TENANT_HOST_PATTERN: "^(.*)-${default-uri}"
resources:
  - name: snapqm-db                   # HDI-Container je Mandant
    type: org.cloudfoundry.managed-service
    parameters: { service: service-manager, service-plan: container }
  - name: snapqm-registry             # SaaS Provisioning Service
    type: org.cloudfoundry.managed-service
    parameters:
      service: saas-registry
      service-plan: application
      config:
        appName: snapqm
        displayName: SnapQM
        appUrls:
          getDependencies: ~{mtx-api/mtx-url}/-/cds/saas-provisioning/dependencies
          onSubscription: ~{mtx-api/mtx-url}/-/cds/saas-provisioning/tenant/{tenantId}
          onSubscriptionAsync: true
```

Listing 4.3: Auszug aus dem MTA-Deskriptor (`mta.yaml`)

4.5 Integration in die Systemlandschaft des Kunden

Eine Micro-App bezieht ihren Nutzen aus der Anbindung an das ERP-System des Kunden. Clean-Core-konform erfolgt sie über freigegebene APIs aus dem SAP Business Accelerator Hub [SA26l]. Deren Metadaten werden mit `cds import` in das Projekt übernommen und stehen danach als typisierter Remote-Service zur Verfügung. Listing 4.4 zeigt die Konfiguration. Entscheidend für den mandantenfähigen Betrieb ist die Auswahlstrategie `alwaysSubscriber`: Die Destination wird stets im Subaccount des aufrufenden Mandanten gesucht. Jeder Kunde pflegt also Adresse und Zugangsdaten seines eigenen Systems, ohne dass der Anbieter sie kennt oder verwaltet.

Listing 4.5 zeigt den Ereignisbehandler, der eine erfasste Prüfung als Qualitätsmeldung an das SAP S/4HANA des Kunden übergibt. Die Abbildung auf Entitäten und Felder der Ziel-API ist vereinfacht; maßgeblich ist das Metadatendokument der jeweils eingesetzten Schnittstelle.

Für Systeme im Rechenzentrum des Kunden stellt der Cloud Connector eine ausgehende, verschlüsselte Verbindung zur BTP her, sodass keine eingehenden Firewall-Freigaben nötig sind [SA26d]. SAP S/4HANA Cloud wird direkt über das Internet angesprochen. Für zeitlich

```
{
  "cds": {
    "profile": "with-mtx-sidecar",
    "requires": {
      "[production]": {
        "multitenancy": true,
        "auth": "xsuaa",
        "db": "hana"
      },
      "API_QUALITYNOTIFICATION": {
        "kind": "odata-v2",
        "model": "srv/external/API_QUALITYNOTIFICATION",
        "credentials": {
          "destination": "SNAPQM_S4",
          "path": "/sap/opu/odata/sap/API_QUALITYNOTIFICATION"
        },
        "destinationOptions": { "selectionStrategy": "alwaysSubscriber" }
      }
    }
  }
}
```

Listing 4.4: Konfiguration von Mandantenfähigkeit und Remote-Service (Auszug aus `package.json`)

entkoppelte Szenarien bieten sich Geschäftsereignisse an, die über SAP Event Mesh oder die SAP Integration Suite zugestellt werden. Weil das Zielsystem nicht unter der Kontrolle des Anbieters steht, müssen Aufrufe robust gegen Ausfälle sein. Das CAP unterstützt dies mit einem persistenten Ausgangspostfach (Outbox), das Nachrichten transaktional speichert und bei Fehlern erneut zustellt [SA26c].

4.6 Sicherheit, Identität und Datenschutz

Authentifizierung und Autorisierung übernimmt der Dienst XSUAA[3]. Im Deskriptor `xs-security.json` definiert der Anbieter Berechtigungen (Scopes) und daraus abgeleitete Rollenvorlagen, im Beispiel „Inspector“ und „QualityManager“. Mit der Einstellung `tenant-mode: shared` wird die Anwendung mandantenfähig; ein zusätzlicher Scope erlaubt dem SaaS Provisioning Service den Aufruf der Callbacks. Der Kunde bündelt die Rollen in seinem Subaccount zu Rollensammlungen und weist sie seinen Nutzern zu. Die Anmeldung erfolgt über SAP Cloud Identity Services, die als Vermittler zum unternehmensweiten Identity Provider des Kunden dienen können (Anforderung T3).

Aus Sicht des Datenschutzes ist der Anbieter Auftragsverarbeiter nach Art. 28 DSGVO [Eu16]. Daraus folgen ein Vertrag zur Auftragsverarbeitung mit jedem Kunden, die Wahl einer Region innerhalb der EU, ein dokumentiertes Lösch- und Exportkonzept für das Offboarding sowie die Prüfung, welche personenbezogenen Daten – etwa Namen in Prüfprotokollen oder Personen auf Fotos – überhaupt gespeichert werden müssen. Die physische Trennung der Mandantendaten in eigenen Containern erleichtert diese Nachweise.

[3] SAP Authorization and Trust Management Service

```
const cds = require('@sap/cds');

module.exports = class InspectionService extends cds.ApplicationService {
  async init() {
    const { Inspections } = this.entities;
    // Remote-Service; die Destination wird im Subaccount des Kunden aufgelöst
    const s4 = await cds.connect.to('API_QUALITYNOTIFICATION');

    this.on('transferToS4', Inspections, async (req) => {
      const { ID } = req.params[0];
      const insp = await SELECT.one.from(Inspections, ID);
      if (insp.status !== 'SUBMITTED') return req.reject(409, 'INSPECTION_NOT_SUBMITTED'
          );

      // Vereinfachte Abbildung; Entitäts- und Feldnamen gemäß $metadata der API
      const qn = await s4.send('POST', '/A_QualityNotification', {
        NotificationType: 'Q2',           // Mängelrüge gegen Lieferant
        Material: insp.material,
        Supplier: insp.supplier,
        NotificationText: `SnapQM WE ${insp.deliveryNo}`
      });
      await UPDATE(Inspections, ID).with({
        status: 'TRANSFERRED', s4NotificationID: qn.QualityNotification
      });
      return qn.QualityNotification;
    });

    return super.init();
  }
};
```

Listing 4.5: Ereignisbehandler für die Übertragung an SAP S/4HANA (`srv/service.js`)

4.7 Entwicklungs-, Auslieferungs- und Betriebsprozesse

Weil eine mandantenfähige Anwendung für alle Kunden gleichzeitig aktualisiert wird, ist ein hoher Automatisierungsgrad in Test und Auslieferung Voraussetzung für einen verlässlichen Betrieb [HF10]. Abbildung 4.4 zeigt den Prozess. Änderungen werden über Pull Requests geprüft, die Pipeline baut mit `cds build` und dem Cloud MTA Build Tool ein Archiv, führt Tests und Sicherheitsprüfungen aus und übergibt das Archiv an SAP Cloud Transport Management, das es über Test- in die Produktivumgebung befördert. Ein abschließender Schritt aktualisiert die Schemata aller Mandanten.

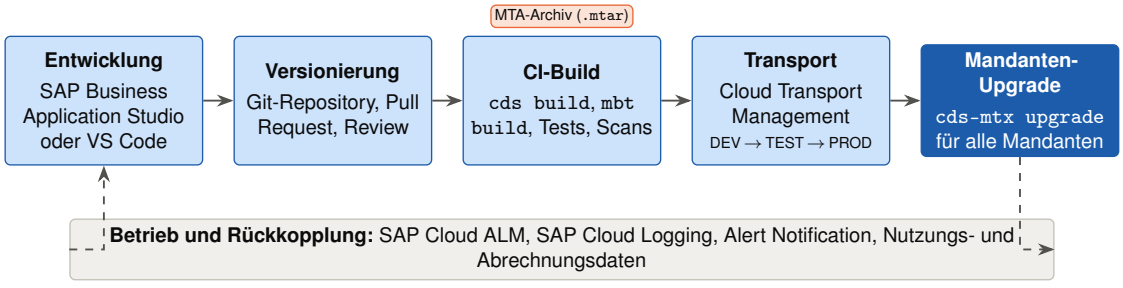


Abbildung 4.4: Entwicklungs-, Auslieferungs- und Betriebsprozess einer Micro-App (eigene Darstellung)

Im Betrieb sind Protokolle und Metriken nach Mandanten auszuwerten, damit Störungen einzelnen Kunden zugeordnet und Nutzungsdaten für die Abrechnung erhoben werden können (Anforderungen T6 und Ö2). Auf der BTP stehen dafür unter anderem SAP Cloud Logging,

der SAP Alert Notification Service und SAP Cloud ALM zur Verfügung. Als Zielgrößen für die Leistungsfähigkeit des Auslieferungsprozesses eignen sich die von Forsgren; Humble; Kim [FHK18] beschriebenen Kennzahlen Bereitstellungshäufigkeit, Durchlaufzeit von Änderungen, Fehlerrate von Änderungen und Wiederherstellungszeit.

4.8 Zwischenfazit: technische Kostentreiber

Die Referenzarchitektur erfüllt die technischen Anforderungen T1 bis T7 weitgehend mit Standardmitteln von Plattform und Framework. Für die ökonomische Betrachtung sind vier Ergebnisse festzuhalten. Erstens entstehen durch HANA Cloud, die Anwendungsmodule und die Entwicklungsumgebungen Fixkosten, die unabhängig von der Kundenzahl anfallen. Zweitens sind die variablen Plattformkosten je Mandant gering, weil Prozesse und Datenbankinstanz geteilt werden und nur Container und Speicher je Mandant hinzukommen. Drittens senkt das automatisierte Onboarding den Aufwand je Neukunde und ermöglicht Self-Service-Modelle. Viertens verlagert die Architektur einen Teil der Wartung auf die Anpassung an Änderungen von Plattform, Framework und SAP-Schnittstellen, die der Anbieter nicht selbst steuert. Diese Größen bilden die Brücke zum Kosten- und Erlösmodell im folgenden Kapitel.

5 Ökonomische Aspekte

Dieses Kapitel entwickelt das Kosten- und Erlösmodell für Micro-Apps auf der BTP. Es beschreibt die Kostenarten über den Lebenszyklus, die kommerziellen Modelle der Plattform, die Skaleneffekte der Mandantenfähigkeit, Preismodelle und Vertriebswege sowie die wichtigsten ökonomischen Risiken. Die Zahlenbeispiele beruhen auf den Annahmen des Basisszenarios der Fallstudie (Anhang A.1).

5.1 Kostenstruktur über den Lebenszyklus

Tabelle 5.1 ordnet die Kosten einer Micro-App nach Kostenart, Treiber und Verhalten gegenüber der Kundenzahl. Aufwände sind in Personentagen (PT[1]) angegeben. Die Einteilung folgt der üblichen Unterscheidung in einmalige, fixe und variable Kosten [BDH15] und ergänzt sie um die umsatzabhängigen Provisionen, die beim Vertrieb über Marktplätze und Partner entstehen.

Tabelle 5.1: Kostenarten einer Micro-App auf der SAP BTP (eigene Darstellung; Werte: Annahmen des Basisszenarios)

Kostenart	Treiber	Verhalten	Basisszenario
Erstentwicklung	Funktionsumfang, Automatisierungsgrad	einmalig	150 PT
Wartung, Weiterentwicklung	Release-Takt von SAP und CAP, Ausbau	fix je Jahr	35 PT je Jahr
Plattform (fix)	HANA-Cloud-Grundinstanz, Laufzeit, DEV/TEST	fix je Jahr	22.800 € je Jahr
Plattform (variabel)	Container und Speicher je Mandant	je Mandant	300 € je Jahr
Betrieb und Support	Anfragen, Störungen, Onboarding-Hilfe	je Mandant	0,6 PT je Jahr
Marketing und Vertrieb	Listing, Veranstaltungen, Vorvertrieb	fix und je Neukunde	30.000 € + 4.000 € je Neukunde
Provision	Umsatz über Marktplatz und Partner	umsatzabhängig	15 %
Partnerprogramm	Mitgliedschaft, Zertifizierung, Listing	fix bzw. einmalig	3.000 € je Jahr, 12.000 € einmalig

[1] Personentag

Die Erstentwicklung lässt sich mit parametrischen Verfahren wie COCOMO II schätzen [Bo00]. Für kleine Vorhaben mit überschaubarem Funktionsumfang ist eine Expertenschätzung je Arbeitspaket jedoch verbreiteter und transparenter; sie wird in Abschnitt 6.2 verwendet. Bemerkenswert ist das Verhältnis der Kostenarten: Bei einer Micro-App sind Entwicklung und Plattform vergleichsweise günstig, weil Framework und Plattform viele Funktionen mitbringen. Vertrieb und Kundengewinnung fallen dagegen in ähnlicher Höhe an wie bei größeren Produkten, weil auch eine kleine Lösung Vertrauen, Sicherheitsprüfungen und Einkaufsprozesse beim Kunden durchlaufen muss [Cu04].

5.2 Plattformkosten und kommerzielle Modelle der SAP BTP

SAP bietet die BTP in verbrauchsbasierten und subskriptionsbasierten Modellen an [SA26e]. Bei den verbrauchsbasierten Modellen erwirbt der Kunde Zugang zu allen dafür freigegebenen Diensten und kann sie während der Vertragslaufzeit flexibel nutzen. Tabelle 5.2 fasst die Varianten aus Sicht eines Anbieters zusammen. Die SAP BTP Enterprise Agreement genannte Variante wurde im April 2024 als Weiterentwicklung des Cloud Platform Enterprise Agreement eingeführt [SA26g].

Tabelle 5.2: Kommerzielle Modelle der SAP BTP aus Anbietersicht (eigene Darstellung nach [SA26e; SA26g])

Modell	Funktionsweise	Bewertung für Micro-Apps
Pay-As-You-Go	keine Mindestabnahme, monatliche Abrechnung des Verbrauchs	geringes Risiko in der Startphase, keine Mengenrabatte
BTPEA / CPEA	vorausbezahlte Cloud Credits mit jährlicher Verbrauchszusage	Rabatte bei höherem Volumen, aber Kapitalbindung und Prognoserisiko
Subskription	feste Gebühr für definierte Dienste und Kapazitäten	gut planbar, aber wenig flexibel bei schwankendem Wachstum

Für einen Anbieter, der mit wenigen Kunden startet, ist ein nutzungsabhängiges Modell ohne Mindestabnahme naheliegend; mit wachsendem und besser prognostizierbarem Verbrauch kann ein Wechsel auf ein Modell mit Verbrauchszusage sinnvoll sein. Die Kosten einzelner Dienste lassen sich mit dem Schätzwerkzeug im SAP Discovery Center ermitteln [SA26k; SA26o]. Eine Herausforderung bleibt die Zuordnung der Kosten zu Mandanten (Anforderung Ö5): Arbeitsspeicher der Laufzeit und Kapazität der Datenbank werden von allen Mandanten gemeinsam genutzt und erscheinen auf der Rechnung nicht je Kunde. Der Anbieter muss daher eigene Verteilungsschlüssel bilden, etwa nach Speicherbelegung der Container und nach Anfragevolumen aus den Protokollen.

5.3 Skaleneffekte durch Mandantenfähigkeit

Wie stark die Architektur die Stückkosten beeinflusst, zeigt ein Vergleich zweier Betriebsmodelle. Im *Single-Tenant-Modell* erhält jeder Kunde eine eigene Installation mit eigener Datenbank- und Laufzeitinstanz; dafür werden 800 € Plattformkosten je Monat und zwei Personentage Aufwand für Bereitstellung und Upgrades je Jahr angesetzt. Im *Multi-Tenant-Modell* der Referenzarchitektur teilen sich alle Kunden eine Installation; neben den fixen Plattformkosten fallen nur die variablen Kosten je Mandant an. Abbildung 5.1 zeigt die jährlichen Betriebskosten je Kunde ohne Vertriebskosten.

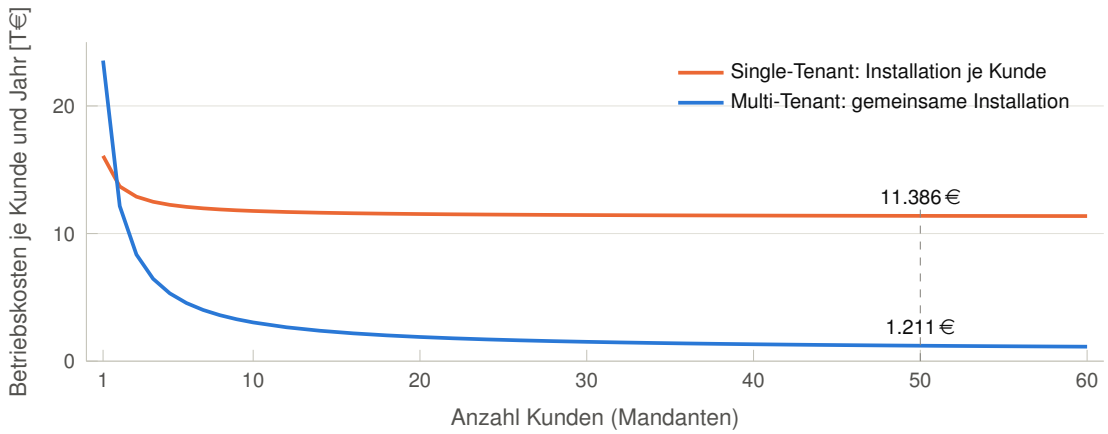


Abbildung 5.1: Jährliche Betriebskosten je Kunde bei Single- und Multi-Tenant-Betrieb (eigene Berechnung)

Bei einem einzigen Kunden ist der Multi-Tenant-Betrieb teurer, weil die gesamte Grundausstattung von einem Kunden getragen wird. Schon ab dem zweiten Kunden kehrt sich das Verhältnis um. Bei zehn Kunden betragen die Betriebskosten je Kunde im Multi-Tenant-Modell 3.035 € gegenüber 11.770 € im Single-Tenant-Modell, bei fünfzig Kunden 1.211 € gegenüber 11.386 €. Die Kosten sinken damit um rund eine Größenordnung, während sie im Single-Tenant-Modell nahezu konstant bleiben. Erst diese Kostendegression erlaubt Preise, die für kleine Kunden attraktiv sind, und erfüllt Anforderung Ö1. Umgekehrt zeigt die Abbildung, dass sich Mandantenfähigkeit nur lohnt, wenn eine ausreichende Zahl an Kunden absehbar ist – für eine Einzellösung bei einem Kunden ist sie unnötiger Aufwand.

5.4 Preis- und Erlösmodelle

Die Wahl der Preisbezugsgröße gehört zu den wirkungsvollsten Entscheidungen eines Softwareanbieters [KC09]. Für Micro-Apps kommen vor allem vier Modelle in Betracht: ein Preis je Nutzer, eine Pauschale je Mandant, eine Paketstaffel nach Nutzerbändern und ein nutzungsabhängiger Preis, etwa je erfasstem Vorgang. Abbildung 5.2 vergleicht die ersten drei Modelle für die Preispunkte der Fallstudie.

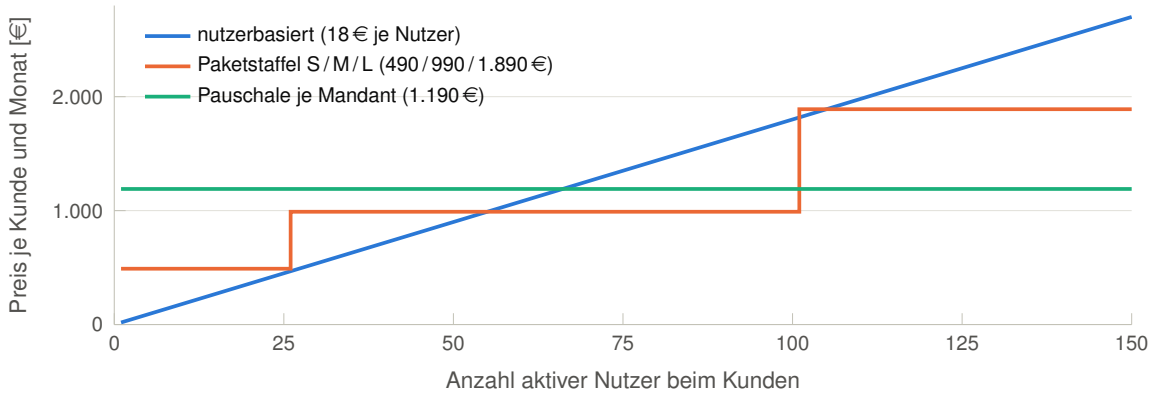


Abbildung 5.2: Monatlicher Preis je Kunde in Abhängigkeit der Nutzerzahl (eigene Darstellung)

Ein Nutzerpreis ist einfach zu verstehen, wirkt aber gerade bei Micro-Apps oft gegen den Nutzen: Eine Erfassungsanwendung im Wareneingang wird von vielen Gelegenheitsnutzern verwendet, und ein Preis je Nutzer verleitet den Kunden dazu, Zugänge zu teilen oder die Nutzung zu begrenzen. Eine Pauschale ist für große Kunden günstig und für kleine zu teuer. Die Paketstaffel bildet einen Kompromiss: Sie folgt grob der Zahlungsbereitschaft, bleibt für den Kunden planbar und verursacht kaum Abrechnungsaufwand, da nur die Nutzerzahl je Mandant erfasst werden muss. Nutzungsabhängige Preise koppeln Preis und Nutzen am engsten, sind für Kunden aber schwerer zu budgetieren und erfordern eine verlässliche Messung (Anforderung Ö2).

Die Höhe des Preises sollte sich am Kundennutzen orientieren und nicht an den Kosten [SF16]. Eine einfache Beispielrechnung verdeutlicht das: Spart die Micro-App in einem Werk mit 4.000 Wareneingangsprüfungen im Jahr je Prüfung zwölf Minuten, entspricht das 800 Arbeitsstunden. Bei einem Stundensatz von 45 € ergibt sich ein jährlicher Nutzen von 36.000 €; das Paket M mit 11.880 € im Jahr schöpft davon rund ein Drittel ab. Nicht eingerechnet sind schwerer messbare Effekte wie schnellere Reklamationen beim Lieferanten. Ergänzend empfiehlt sich ein zeitlich begrenzter Testzugang, der dank automatisiertem Onboarding ohne Aufwand für den Anbieter bereitgestellt werden kann und die Akquisitionskosten senkt [Ri11].

5.5 Vertrieb und Partnerschaft mit SAP

Der Vertrieb ist bei Micro-Apps der kritische Engpass, weil der erzielbare Umsatz je Kunde begrenzt ist und hohe Akquisitionskosten die Marge schnell aufzehren. Drei Kanäle stehen zur Verfügung. Der *Direktvertrieb* erlaubt die engste Kundenbeziehung, ist aber für kleine Anbieter teuer. *Implementierungspartner und Systemhäuser* haben bereits Zugang zu SAP-Kunden und können die Micro-App in Projekte einbringen, erwarten dafür aber eine Beteili-

gung. Der *SAP Store* ist der Marktplatz für Partnerlösungen; SAP gibt an, Partnern darüber Sichtbarkeit bei mehr als 440.000 SAP-Kunden zu verschaffen [SA26b].

Der Zugang zum SAP Store ist an das Partnerprogramm SAP PartnerEdge, Build gebunden. Es umfasst Entwicklungs- und Testzugänge, Unterstützung bei der Markteinführung sowie die Zertifizierung der Lösung [SA26b]. Für den Anbieter entstehen Programmgebühren, Aufwand für Zertifizierung und Listing sowie gegebenenfalls Provisionen, deren Höhe vertraglich geregelt wird; im Modell sind sie mit den Werten aus Tabelle 5.1 angesetzt. Dem stehen Vorteile gegenüber, die in der Literatur belegt sind: Die Partnerschaft signalisiert Qualität und Kompatibilität und verbessert den Marktzugang, was sich in höheren Umsätzen niederschlägt [Ce12].

5.6 Ökonomische Risiken

Mehrere Risiken sind für Micro-Apps auf der BTP spezifisch. Das erste ist die *Plattformabhängigkeit*. Eine Anwendung, die tief in Dienste wie XSUAA, SaaS Provisioning und HANA Cloud integriert ist, lässt sich nur mit hohem Aufwand auf eine andere Plattform übertragen [OST16]. Preisänderungen oder Abkündigungen von Diensten treffen den Anbieter unmittelbar. Das zweite Risiko ist der *Eintritt des Plattformbetreibers*: Erweitert SAP den Standard um die Funktion der Micro-App, verliert diese ihren Wert. Huang et al. [Hu13] zeigen, dass Anbieter dieses Risiko bei der Partnerschaftsentscheidung abwägen. Micro-Apps mit spezifischem Domänenwissen, etwa für eine Branche oder eine Nische, sind weniger gefährdet als generische Funktionen. Das dritte Risiko liegt in *langen Vertriebszyklen*: Auch eine günstige Lösung durchläuft beim Kunden Sicherheits- und Datenschutzprüfungen, was den Umsatz verzögert und die Vorfinanzierung verlängert. Schließlich hängt die Micro-App von *Stabilität und Umfang der SAP-Schnittstellen* ab. Fehlt eine benötigte API oder ändert sie sich, entstehen Wartungskosten, die der Anbieter nicht beeinflussen kann.

6 Fallstudie: Micro-App „SnapQM“

Die Fallstudie demonstriert Referenzarchitektur und Kosten- und Erlösmodell an einer konkreten Micro-App. Das Szenario ist fiktiv, aber an typischen Abläufen in fertigenden Unternehmen orientiert. Alle Zahlen sind Modellannahmen; sie sind in Anhang A.1 vollständig aufgeführt und werden durch das Skript in Anhang A.2 reproduzierbar berechnet.

6.1 Ausgangssituation und Anwendungsfall

In vielen mittelständischen Fertigungsunternehmen wird die Wareneingangsprüfung noch auf Papier oder in Tabellen dokumentiert. Stellt ein Prüfer an der Rampe einen Mangel fest, fotografiert er ihn mit einem privaten oder geteilten Mobilgerät, schickt die Bilder per E-Mail an das Qualitätswesen und notiert die Befunde auf dem Lieferschein. Erst später legt ein Sachbearbeiter in SAP S/4HANA eine Qualitätsmeldung vom Typ „Mängelrüge gegen Lieferant“ an und überträgt die Angaben von Hand. Der Ablauf ist langsam, fehleranfällig und verzögert die Reklamation beim Lieferanten.

Die Micro-App *SnapQM* unterstützt genau diese eine Aufgabe (Abbildung 6.1). Der Prüfer sieht auf einem Mobilgerät die offenen Wareneingänge seines Werks, erfasst Mängel mit Fehlerart, Menge und Foto und schließt die Prüfung ab. Ein Qualitätsverantwortlicher gibt sie frei; die Micro-App legt daraufhin die Qualitätsmeldung im SAP S/4HANA des Kunden an und speichert deren Nummer. Damit erfüllt SnapQM die Arbeitsdefinition aus Abschnitt 2.1: eine Aufgabe, zwei klar umrissene Rollen, ein vertikaler Schnitt und eine Anbindung über eine freigegebene Schnittstelle.

6.2 Umsetzung auf Basis der Referenzarchitektur

SnapQM wird nach der Referenzarchitektur aus Kapitel 4 aufgebaut; die dort gezeigten Listings sind Auszüge aus diesem Entwurf. Tabelle 6.1 zeigt die Aufwandsschätzung für ein MVP[1], das alle Anforderungen aus Tabelle 3.1 in einer ersten Ausbaustufe erfüllt. Die Schät-

[1] Minimum Viable Product

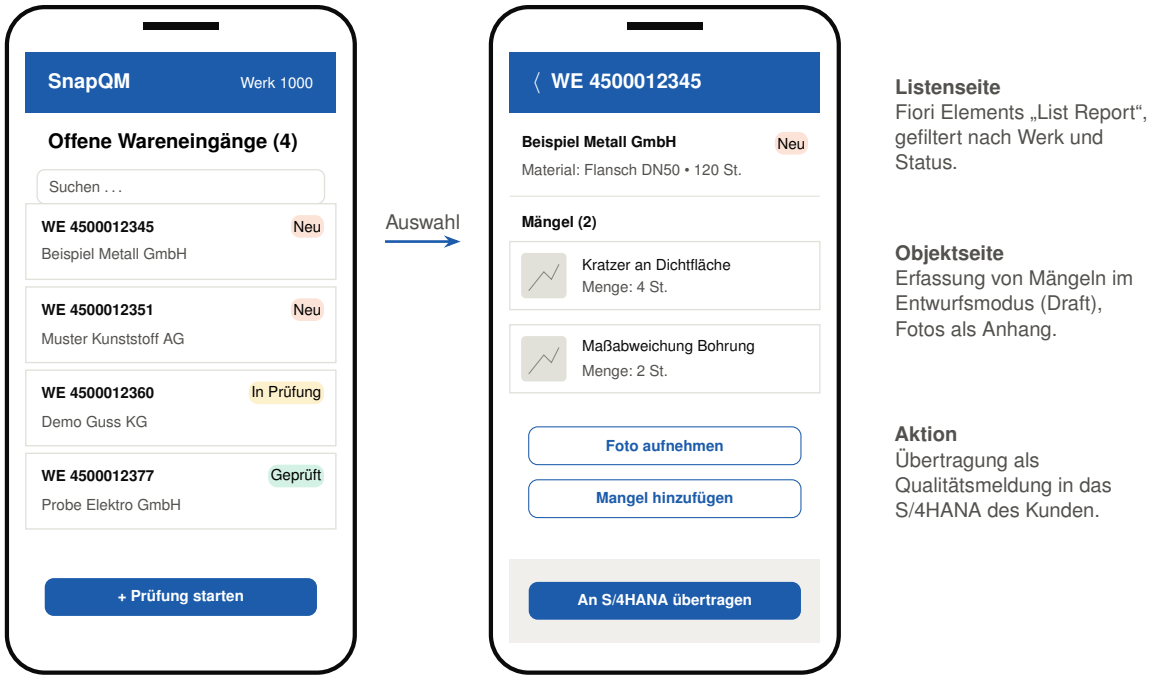


Abbildung 6.1: Oberflächenentwurf der Micro-App SnapQM mit Listen- und Objektseite (eigene Darstellung, alle Daten fiktiv)

zung erfolgte je Arbeitspaket als Expertenschätzung und ist entsprechend unsicher; sie geht in der Sensitivitätsanalyse mit einer Bandbreite von $\pm 20\,\%$ ein.

Tabelle 6.1: Aufwandsschätzung für das MVP von SnapQM (eigene Schätzung)

Arbeitspaket	Aufwand [PT]
Anforderungsanalyse und UX-Konzept	15
Datenmodell und Service-Schicht mit CAP	25
Oberfläche mit SAP Fiori Elements, Fotoerfassung	20
Integration in SAP S/4HANA (Remote-Service, Outbox, Fehlerbehandlung)	20
Mandantenfähigkeit, Onboarding, Rollenkonzept	20
Sicherheit, Datenschutz, Lösch- und Exportkonzept	10
CI/CD, Transport, Monitoring	10
Test und Qualitätssicherung	20
Dokumentation und Vorbereitung der Zertifizierung	10
Summe	**150**

Auf Mandantenfähigkeit und den automatisierten Lebenszyklus entfallen etwa 20 bis 25 Personentage und damit rund ein Siebtel des Gesamtaufwands. Dieser Mehraufwand ist gering, weil das CAP die wesentlichen Funktionen bereitstellt; er amortisiert sich nach Abschnitt 5.3 bereits ab wenigen Kunden. Bei einem internen Vollkostensatz von 650 € je Personentag ergeben sich Entwicklungskosten von 97.500 €. Zusammen mit Zertifizierung und Listing sowie Plattformkosten der Entwicklungsphase beträgt die Anfangsinvestition 114.300 €.

6.3 Geschäftsmodell

Abbildung 6.2 fasst das Geschäftsmodell im Business Model Canvas zusammen. Zielgruppe sind fertigende Mittelständler in der DACH-Region, die SAP S/4HANA einsetzen. Das Wertangebot besteht in einer schnelleren und lückenlosen Wareneingangsprüfung ohne Eingriff in das ERP-System. Vertrieben wird SnapQM über den SAP Store und über Systemhäuser; der Kunde kann einen Testmandanten selbst anlegen.



Abbildung 6.2: Business Model Canvas der Micro-App SnapQM (eigene Darstellung nach [OP10])

Gemäß Abschnitt 5.4 wird eine Paketstaffel je Mandant verwendet: Paket S für bis zu 25 Nutzer zu 490 €, Paket M für bis zu 100 Nutzer zu 990 € und Paket L ohne Nutzergrenze zu 1.890 € je Monat. Bei einer angenommenen Verteilung von 50 %, 35 % und 15 % auf die drei Pakete ergibt sich ein durchschnittlicher Umsatz je Kunde von 875 € im Monat beziehungsweise 10.500 € im Jahr.

6.4 Wirtschaftlichkeitsanalyse

Die Analyse betrachtet ein Entwicklungsjahr (Jahr 0) und fünf Betriebsjahre. Im Basisszenario gewinnt der Anbieter 5, 12, 18, 22 und 25 Neukunden pro Jahr, jährlich kündigen 10 % des Bestands. Umsätze und kundenabhängige Kosten werden auf den durchschnittlichen Kundenbestand des Jahres bezogen; die Kundenzahlen sind Erwartungswerte und daher nicht ganzzahlig. Die Rechnung erfolgt vor Steuern und ohne Restwert am Ende des Betrachtungszeitraums; der Kalkulationszins von 10 % berücksichtigt das Risiko eines neuen Produkts. Tabelle 6.2 zeigt die Planrechnung.

Tabelle 6.2: Planrechnung für SnapQM im Basisszenario in T€ (eigene Berechnung)

	Jahr 0	Jahr 1	Jahr 2	Jahr 3	Jahr 4	Jahr 5
Neukunden	0	5	12	18	22	25
Kunden (Jahresende)	0,0	5,0	16,5	32,9	51,6	71,4
Umsatzerlöse	0,0	26,2	112,9	259,1	443,2	645,6
Entwicklung und Wartung	97,5	22,8	22,8	22,8	22,8	22,8
Plattformkosten BTP	4,8	23,6	26,0	30,2	35,5	41,2
Vertrieb, Marketing, Partner	12,0	56,9	97,9	143,9	187,5	229,8
Betrieb und Support	0,0	1,0	4,2	9,6	16,5	24,0
Summe Auszahlungen	114,3	104,2	150,9	206,4	262,1	317,8
Netto-Cashflow	−114,3	−78,0	−38,0	52,6	181,0	327,8
Kumulierter Cashflow	−114,3	−192,3	−230,3	−177,6	3,4	331,2

Der kumulierte Cashflow wird im vierten Betriebsjahr positiv; die statische Amortisationsdauer beträgt 4,0 Jahre. Am Ende des fünften Jahres hat SnapQM 71 Kunden und einen Jahresumsatz von 645,6 T€. Der Kapitalwert beträgt 150,1 T€, die Investition ist im Basisszenario also vorteilhaft. Abbildung 6.3 zeigt, wie sich die Auszahlungen zusammensetzen. Ab dem ersten Betriebsjahr dominieren Vertrieb, Marketing und Provisionen; im fünften Jahr machen sie gut 70 % der Auszahlungen aus. Die Plattformkosten der BTP liegen dagegen bei rund 13 %, die laufende Entwicklung bei unter 10 %.

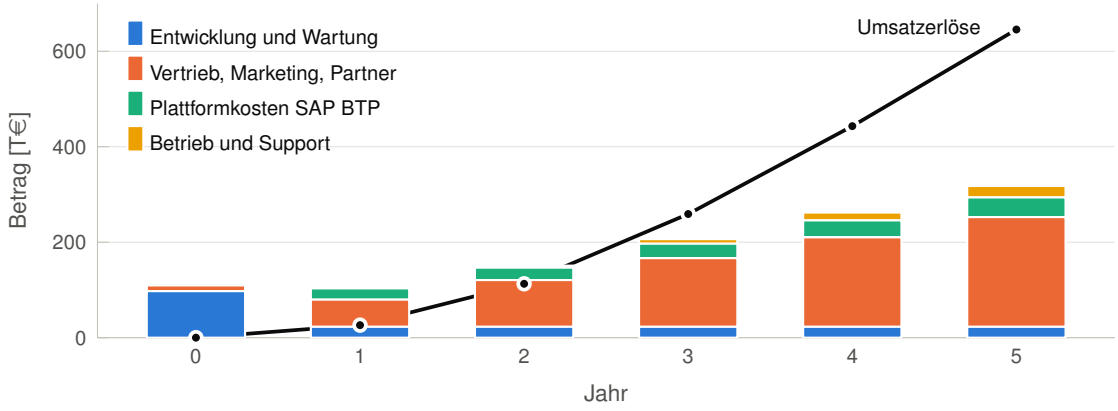


Abbildung 6.3: Auszahlungen nach Kostenart und Umsatzerlöse im Basisszenario (eigene Berechnung)

Die Kennzahlen auf Kundenebene bestätigen dieses Bild. Nach Abzug von Provision, variablen Plattformkosten und Support verbleibt je Kunde ein jährlicher Deckungsbeitrag von 8.235 €, das entspricht 78,4 % des Umsatzes. Mit dem Kalkulationszins und der Kündigungsquote abgezinst ergibt sich ein Kundenwert (LTV) von 41.175 €. Die Akquisitionskosten betragen im fünften Jahr 5.200 € je Neukunde, wenn die fixen Marketingkosten auf die Neukunden verteilt werden. Das Verhältnis LTV/CAC liegt damit bei 7,9, und der Deckungsbeitrag eines Kunden hat seine Akquisitionskosten nach 7,6 Monaten eingespielt. Beide Werte liegen deutlich über den Richtwerten aus Tabelle 2.2. In den ersten Jahren ist das Verhältnis

allerdings ungünstiger, weil sich die fixen Marketingkosten auf wenige Neukunden verteilen: Im ersten Betriebsjahr betragen die Akquisitionskosten 10.000 € je Kunde.

6.5 Szenario- und Sensitivitätsanalyse

Um die Unsicherheit der Annahmen abzubilden, werden zwei weitere Szenarien gerechnet. Im *pessimistischen Szenario* gewinnt der Anbieter 40 % weniger Neukunden, die Kündigungsquote steigt auf 15 %, die Preise sinken um 10 % und die Erstentwicklung dauert 25 % länger. Im *optimistischen Szenario* liegen die Neukunden 30 % über dem Basisszenario, die Kündigungsquote bei 6 %, die Preise 10 % höher und der Entwicklungsaufwand 10 % niedriger. Tabelle 6.3 und Abbildung 6.4 zeigen die Ergebnisse.

Tabelle 6.3: Ergebnisse der Szenarioanalyse (eigene Berechnung)

	pessimistisch	Basis	optimistisch
Kunden nach 5 Jahren	40	71	98
Umsatz Jahr 5 [T€]	326,6	645,6	966,1
Kapitalwert ($i = 10\,\%$) [T€]	−170,3	150,1	473,7
Amortisationsdauer [Jahre]	> 5	4,0	3,2
LTV/CAC-Verhältnis	4,9	7,9	11,6
CAC-Payback [Monate]	9,8	7,6	6,5

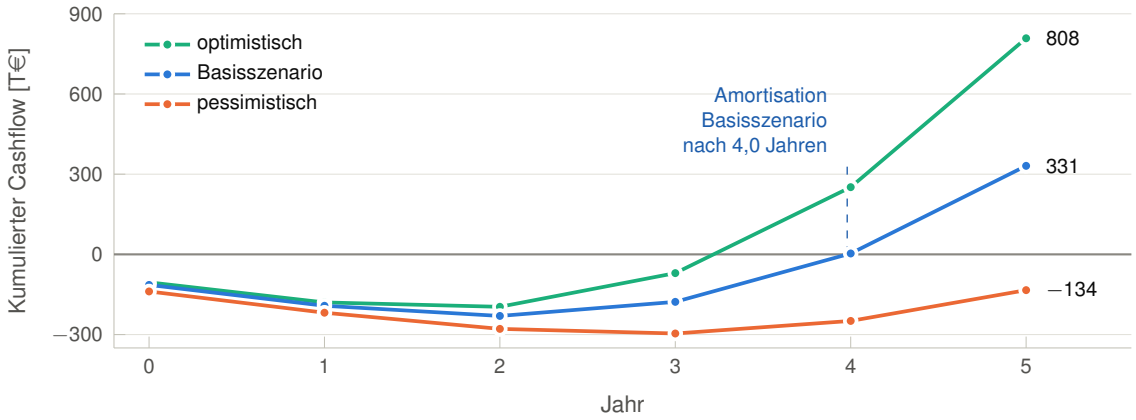


Abbildung 6.4: Kumulierter Netto-Cashflow in den drei Szenarien (eigene Berechnung)

Die Bandbreite ist erheblich. Im pessimistischen Szenario amortisiert sich die Investition innerhalb von fünf Jahren nicht, der Kapitalwert beträgt −170,3 T€. Im optimistischen Szenario steigt er auf 473,7 T€. Bemerkenswert ist, dass selbst das pessimistische Szenario ein LTV/CAC-Verhältnis über 3 aufweist. Das Problem liegt dort nicht in der Wirtschaftlichkeit des einzelnen Kunden, sondern darin, dass zu wenige Kunden gewonnen werden, um Fixkosten und Anfangsinvestition in fünf Jahren zu decken.

Die Sensitivitätsanalyse in Abbildung 6.5 variiert jeweils einen Parameter um ±20 % bei sonst unveränderten Annahmen. Den größten Einfluss hat das *Preisniveau*: Sinkt es um

20 %, fällt der Kapitalwert auf −22,5 T€, steigt es um 20 %, erreicht er 322,7 T€. Es folgen die Zahl der *Neukunden* und die *Akquisitionskosten je Kunde*. Technische Parameter wie Entwicklungsaufwand und fixe Plattformkosten wirken deutlich schwächer. Die geringe Wirkung der Kündigungsquote ist dem kurzen Betrachtungszeitraum geschuldet; über einen längeren Zeitraum würde sie an Bedeutung gewinnen.

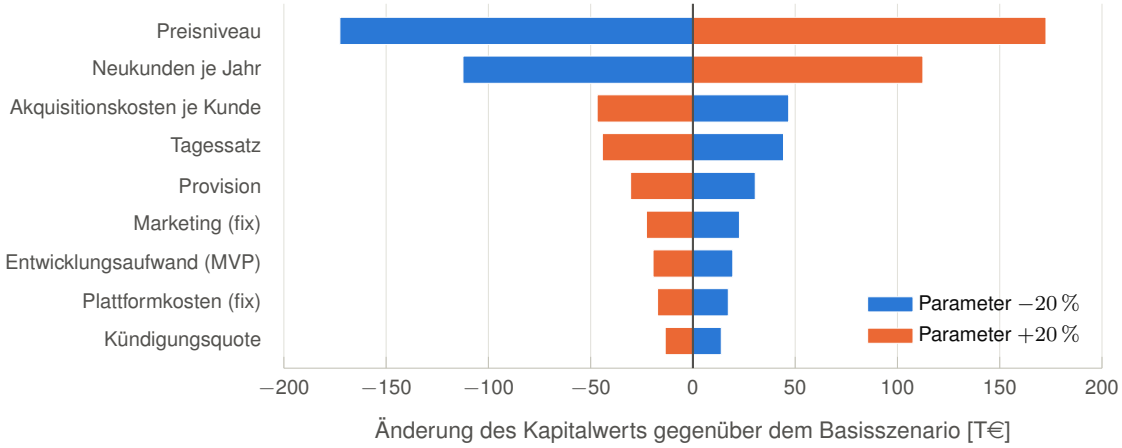


Abbildung 6.5: Sensitivität des Kapitalwerts bei Variation einzelner Parameter um ±20 %; Kapitalwert im Basisszenario 150,1 T€ (eigene Berechnung)

Das Ergebnis deckt sich mit dem Befund von Simon; Fassnacht [SF16], dass der Preis der stärkste Gewinnhebel ist. Für die Fragestellung dieser Arbeit ist aber eine zweite Beobachtung wichtiger: Die direkten Kosten der technischen Umsetzung bestimmen die Wirtschaftlichkeit kaum. Technische Entscheidungen wirken vor allem indirekt – indem Mandantenfähigkeit niedrige Grenzkosten und damit attraktive Preise ermöglicht und indem automatisiertes Onboarding und Testzugänge die Akquisitionskosten senken.

7 Diskussion

7.1 Beantwortung der Forschungsfragen

FF1: Merkmale und Anforderungen. Eine kommerzialisierbare Micro-App unterstützt genau eine Geschäftsaufgabe für eine klar umrissene Rolle, wird als vertikaler Schnitt eigenständig bereitgestellt und ist selbst die Einheit, die angeboten und abgerechnet wird. Kommerzialisierbar wird sie, wenn sie ohne kundenspezifischen Code bei vielen Kunden einsetzbar ist, eine messbare Preisbezugsgröße besitzt und mit geringen Grenzkosten betrieben werden kann. Der Anforderungskatalog in Tabelle 3.1 übersetzt diese Merkmale in sieben technische, fünf ökonomische und zwei rechtlich-organisatorische Anforderungen, die über Abhängigkeiten wie Automatisierung und Akquisitionskosten miteinander verbunden sind.

FF2: Architektur- und Technologieentscheidungen. Für einen mandantenfähigen und wirtschaftlichen Betrieb eignet sich die Cloud-Foundry-Umgebung mit dem CAP besonders, weil das Framework Mandantentrennung, Mandantenlebenszyklus und Erweiterbarkeit als Standardfunktionen anbietet. Die zentralen Entscheidungen der Referenzarchitektur sind ein HDI-Container je Mandant in einer gemeinsamen Datenbank, ein vollständig automatisiertes Onboarding über SaaS Provisioning Service und MTX-Sidecar, die Integration über freigegebene APIs mit Destinations im Subaccount des Kunden, eine mandantenfähige Berechtigungsverwaltung mit XSUAA sowie eine Pipeline, die neue Versionen einschließlich Schema-Upgrades für alle Mandanten ausrollt. Mandantenspezifische Erweiterungen sollten zurückhaltend eingesetzt werden, weil jede Variante die Wartungskosten erhöht.

FF3: Kosten- und Erlösstrukturen. Die Kostenstruktur einer Micro-App ist durch eine überschaubare Anfangsinvestition, nennenswerte Fixkosten für Plattform und Wartung, geringe variable Betriebskosten je Mandant und hohe Vertriebskosten geprägt. Mandantenfähigkeit senkt die Betriebskosten je Kunde gegenüber Einzelinstallationen um rund eine Größenordnung und ist damit die Voraussetzung für Preise, die auch für kleinere Kunden tragbar sind. Über die Wirtschaftlichkeit entscheiden in der Fallstudie jedoch vor allem Preisniveau, Kundengewinnung und Akquisitionskosten; Entwicklungsaufwand und Plattformkosten haben einen deutlich geringeren Einfluss.

7.2 Wirkungszusammenhänge und Handlungsempfehlungen

Abbildung 7.1 verdichtet die Ergebnisse zu einer Wirkungskette. Technische Entscheidungen beeinflussen zunächst Kosten- und Erlöstreiber und erst über diese die ökonomischen Kennzahlen. Die Fallstudie zeigt, dass die Pfade über Grenzkosten, Onboarding-Aufwand und Time-to-Market stärker wirken als der direkte Pfad über die Entwicklungskosten.



Abbildung 7.1: Wirkungskette von technischen Entscheidungen zu ökonomischen Kennzahlen (eigene Darstellung)

Daraus lassen sich Empfehlungen für Anbieter ableiten:

1. **Den Zuschnitt eng halten.** Eine Micro-App sollte eine Aufgabe vollständig lösen, statt viele halb. Ein enger Zuschnitt verkürzt die Entwicklung, schärft das Wertversprechen und erleichtert die Preisfindung.
2. **Mandantenfähigkeit von Anfang an einplanen.** Das Nachrüsten in einem gewachsenen Kundenprojekt ist teurer als der Mehraufwand von rund einem Siebtel bei Neuentwicklung mit dem CAP.
3. **Konfiguration vor Erweiterung.** Kundenspezifische Anforderungen sollten über Einstellungen und Wertelisten abgebildet werden. Mandantenspezifische Erweiterungen des Datenmodells sind die Ausnahme.
4. **Den Lebenszyklus vollständig automatisieren.** Automatisiertes Onboarding, Testzugänge und Upgrades sind keine technische Kür, sondern senken unmittelbar die Akquisitions- und Betriebskosten.
5. **Nutzung messen, bevor sie bepreist wird.** Protokollierung und Metriken je Mandant sind die Grundlage für Kostenzuordnung, Preismodell und Produktentscheidungen.
6. **Den Preis am Nutzen ausrichten.** Paketstaffeln nach Nutzerbändern verbinden Planbarkeit für den Kunden mit Abschöpfung der Zahlungsbereitschaft. Preisuntergrenzen ergeben sich aus den Stückkosten, Preisobergrenzen aus dem Kundennutzen.

7. **Den Vertrieb skalierbar machen.** SAP Store, Partnerprogramm und Systemhäuser verringern die Akquisitionskosten, kosten aber Gebühren und Provisionen. Ihr Nutzen sollte an der Entwicklung der CAC gemessen werden.
8. **Plattformrisiken aktiv steuern.** Abhängigkeiten von einzelnen Diensten sind zu dokumentieren, die Roadmap von SAP ist zu verfolgen, und die Micro-App sollte Domänenwissen bündeln, das SAP nicht ohne Weiteres in den Standard übernimmt.

7.3 Limitationen

Die Ergebnisse unterliegen mehreren Einschränkungen. Erstens beruhen alle Zahlen der Fallstudie auf Annahmen und nicht auf erhobenen Daten eines realen Anbieters; Szenario- und Sensitivitätsanalyse zeigen die Bandbreite, ersetzen aber keine empirische Validierung. Zweitens ändern sich Preise und Konditionen der BTP und der Partnerprogramme regelmäßig, sodass absolute Werte schnell veralten. Die qualitativen Zusammenhänge, insbesondere die Kostendegression durch Mandantenfähigkeit und die Dominanz der Vertriebskosten, dürften robuster sein. Drittens betrachtet die Arbeit nur einen Fall und konzentriert sich auf Cloud Foundry und das CAP; für Kyma, das ABAP Environment und Low-Code-Ansätze können andere Gewichtungen gelten. Viertens wurde die Referenzarchitektur auf Basis der Dokumentation entworfen. Die Quelltextauszüge sind nicht gegen eine produktive Laufzeitumgebung getestet, und Laufzeitverhalten wie Antwortzeiten oder Ressourcenbedarf je Mandant wurde nicht gemessen. Schließlich wurde die gesamte Arbeit von einem KI-Sprachmodell erstellt, ohne Interviews mit Anbietern oder Kunden; die Einschätzungen in Tabelle 4.1 und die Aufwandsschätzung in Tabelle 6.1 sollten daher durch Fachleute überprüft werden.

8 Fazit und Ausblick

Die Arbeit hat untersucht, unter welchen technischen und ökonomischen Bedingungen sich Micro-Apps auf der BTP als Produkt entwickeln, betreiben und verkaufen lassen. Dazu wurden eine Arbeitsdefinition und ein Anforderungskatalog entwickelt, eine Referenzarchitektur auf Basis von Cloud Foundry und CAP entworfen und ein reproduzierbares Kosten- und Erlösmodell aufgestellt. Die Fallstudie SnapQM hat beide Artefakte zusammengeführt.

Das zentrale Ergebnis lässt sich in zwei Sätzen zusammenfassen. Technisch ist die BTP mit dem CAP gut vorbereitet: Mandantenfähigkeit, automatisiertes Onboarding und Clean-Core-konforme Integration sind mit Standardmitteln und überschaubarem Mehraufwand erreichbar, und sie senken die Betriebskosten je Kunde um rund eine Größenordnung. Ökonomisch entscheidet sich der Erfolg dennoch nicht in der Architektur, sondern im Markt – an Preis, Kundengewinnung und Vertriebskosten. Die Architektur ist damit eine notwendige, aber keine hinreichende Bedingung. Ihr größter ökonomischer Beitrag liegt darin, niedrige Preise und einen kostengünstigen, weitgehend selbstbedienten Vertrieb überhaupt erst zu ermöglichen.

Für weitere Arbeiten ergeben sich mehrere Ansatzpunkte. Naheliegend ist eine empirische Validierung des Modells mit Daten von Anbietern, die Micro-Apps bereits über den SAP Store vertreiben, etwa zu tatsächlichen Akquisitionskosten, Kündigungsquoten und Plattformkosten je Mandant. Ebenso lohnend wäre ein Vergleich mit Kyma und dem ABAP Environment anhand realer Messungen. Inhaltlich gewinnen Funktionen der generativen Künstlichen Intelligenz an Bedeutung, die SAP über den Generative AI Hub und Joule auf der Plattform anbietet. Für Micro-Apps eröffnen sie neue Wertangebote, etwa die automatische Klassifikation von Mängeln auf Fotos, verändern aber auch die Kostenstruktur, weil verbrauchsabhängige Modellaufrufe die variablen Kosten je Mandant erhöhen. Wie sich solche Funktionen in Preis- und Erlösmodelle kleiner Anwendungen integrieren lassen, ist eine offene und praktisch relevante Frage.

Abkürzungsverzeichnis

API	Application Programming Interface	1
ARPA	Average Revenue per Account	10
ARR	Annual Recurring Revenue	10
BTP	SAP Business Technology Platform	1
BTPEA	SAP BTP Enterprise Agreement	25
CAC	Customer Acquisition Cost	10
CAP	SAP Cloud Application Programming Model	2
CDS	Core Data Services	7
CPEA	Cloud Platform Enterprise Agreement	25
DSGVO	Datenschutz-Grundverordnung	13
HDI	SAP HANA Deployment Infrastructure	8
ISV	Independent Software Vendor	1
JWT	JSON Web Token	16
LTV	Customer Lifetime Value	10
MRR	Monthly Recurring Revenue	10
MTA	Multi-Target Application	16
MVP	Minimum Viable Product	29
NIST	National Institute of Standards and Technology	6
OData	Open Data Protocol	7
PaaS	Platform as a Service	6
PT	Personentag	24
RAP	ABAP RESTful Application Programming Model	7
SaaS	Software as a Service	4
SLA	Service Level Agreement	13
URL	Uniform Resource Locator	19
XSUAA	SAP Authorization and Trust Management Service	21

Tabellenverzeichnis

2.1 Merkmale von Micro-Apps im Vergleich (eigene Darstellung) 5
2.2 Zentrale Kennzahlen für SaaS-Geschäftsmodelle 11

3.1 Anforderungskatalog für kommerzialisierbare Micro-Apps (eigene Darstellung) 13

4.1 Vergleich von Laufzeitumgebungen und Programmiermodellen 15

5.1 Kostenarten einer Micro-App auf der SAP BTP (eigene Darstellung; Werte: Annahmen des Basisszenarios) 24
5.2 Kommerzielle Modelle der SAP BTP aus Anbietersicht 25

6.1 Aufwandsschätzung für das MVP von SnapQM (eigene Schätzung) 30
6.2 Planrechnung für SnapQM im Basisszenario in T€ (eigene Berechnung) . . 32
6.3 Ergebnisse der Szenarioanalyse (eigene Berechnung) 33

A.1 Annahmen des Basisszenarios (Annahmen des Autors) ix

Abbildungsverzeichnis

1.1 Aufbau der Arbeit entlang des DSR-Prozesses 3

2.1 Einordnung der Micro-App gegenüber Monolith und Microservice-Architektur 5
2.2 Vereinfachter Überblick über die SAP BTP 6
2.3 Erweiterungsmodelle nach dem Clean-Core-Prinzip 8
2.4 Grundmodelle der Datenhaltung in mandantenfähigen Anwendungen 9
2.5 Wert- und Zahlungsströme im Plattform-Ökosystem der SAP BTP (eigene Darstellung) 10

4.1 Referenzarchitektur einer mandantenfähigen Micro-App 16
4.2 Kontenmodell eines Anbieters mit Provider- und Consumer-Subaccounts (eigene Darstellung) 18
4.3 Ablauf des automatisierten Onboardings eines neuen Mandanten (eigene Darstellung) 19
4.4 Entwicklungs-, Auslieferungs- und Betriebsprozess einer Micro-App (eigene Darstellung) 22

5.1 Betriebskosten je Kunde bei Single- und Multi-Tenant-Betrieb 26
5.2 Monatlicher Preis je Kunde in Abhängigkeit der Nutzerzahl (eigene Darstellung) 27

6.1 Oberflächenentwurf der Micro-App SnapQM 30
6.2 Business Model Canvas der Micro-App SnapQM 31
6.3 Auszahlungen nach Kostenart und Umsatzerlöse im Basisszenario (eigene Berechnung) 32
6.4 Kumulierter Netto-Cashflow in den drei Szenarien (eigene Berechnung) . . 33
6.5 Sensitivität des Kapitalwerts 34

7.1 Wirkungskette von technischen Entscheidungen zu ökonomischen Kennzahlen (eigene Darstellung) 36

Quellcodeverzeichnis

4.1 Domänenmodell der Micro-App in CDS (`db/schema.cds`) 17
4.2 Service-Definition mit Berechtigungen und Aktion (`srv/service.cds`) . . 18
4.3 Auszug aus dem MTA-Deskriptor (`mta.yaml`) 20
4.4 Konfiguration von Mandantenfähigkeit und Remote-Service 21
4.5 Ereignisbehandler für die Übertragung an SAP S/4HANA 22

A.1 Kernfunktion des Berechnungsmodells (`src/wirtschaftlichkeit.py`) . xi

Literatur

[An06] Anderson, C.: The Long Tail: Why the Future of Business Is Selling Less of More. Hyperion, New York, 2006.

[BDH15] Buxmann, P.; Diefenbach, H.; Hess, T.: Die Softwareindustrie: Ökonomische Prinzipien, Strategien, Perspektiven. Springer Gabler, Berlin, Heidelberg, 2015.

[Bo00] Boehm, B. W. et al.: Software Cost Estimation with COCOMO II. Prentice Hall, Upper Saddle River, 2000.

[BZ10] Bezemer, C.-P.; Zaidman, A.: Multi-Tenant SaaS Applications: Maintenance Dream or Nightmare? In: Proceedings of the Joint ERCIM Workshop on Software Evolution (EVOL) and International Workshop on Principles of Software Evolution (IWPSE). ACM, New York, S. 88–92, 2010.

[CC06] Chong, F.; Carraro, G.: Architecture Strategies for Catching the Long Tail, Microsoft Corporation, MSDN Library, 2006.

[Ce12] Ceccagnoli, M. et al.: Cocreation of Value in a Platform Ecosystem: The Case of Enterprise Software. MIS Quarterly 36 (1), S. 263–290, 2012.

[Cu04] Cusumano, M. A.: The Business of Software. Free Press, New York, 2004.

[Eu16] Europäisches Parlament und Rat der Europäischen Union: Verordnung (EU) 2016/679 zum Schutz natürlicher Personen bei der Verarbeitung personenbezogener Daten (Datenschutz-Grundverordnung), Amtsblatt der Europäischen Union, L 119, S. 1–88, 2016.

[FHK18] Forsgren, N.; Humble, J.; Kim, G.: Accelerate: The Science of Lean Software and DevOps. IT Revolution, Portland, 2018.

[GC14] Gawer, A.; Cusumano, M. A.: Industry Platforms and Ecosystem Innovation. Journal of Product Innovation Management 31 (3), S. 417–433, 2014.

[Ge20] Geers, M.: Micro Frontends in Action. Manning, Shelter Island, 2020.

[GH13] Ghazawneh, A.; Henfridsson, O.: Balancing Platform Control and External Contribution in Third-Party Development: The Boundary Resources Model. Information Systems Journal 23 (2), S. 173–192, 2013.

[Gö14] Götze, U.: Investitionsrechnung: Modelle und Analysen zur Beurteilung von Investitionsvorhaben. Springer Gabler, Berlin, Heidelberg, 2014.

[He04] Hevner, A. R. et al.: Design Science in Information Systems Research. MIS Quarterly 28 (1), S. 75–105, 2004.

[HF10] Humble, J.; Farley, D.: Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley, Upper Saddle River, 2010.

[Hu13] Huang, P. et al.: Appropriability Mechanisms and the Platform Partnership Decision: Evidence from Enterprise Software. Management Science 59 (1), S. 102–121, 2013.

[IS23] ISO/IEC: ISO/IEC 25010:2023 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model, Genf: International Organization for Standardization, 2023.

[Ja19] Jackson, C.: Micro Frontends, 2019, https://martinfowler.com/articles/micro-frontends.html, Stand: 16. 10. 2026.

[JCG18] Jacobides, M. G.; Cennamo, C.; Gawer, A.: Towards a Theory of Ecosystems. Strategic Management Journal 39 (8), S. 2255–2276, 2018.

[KC09] Kittlaus, H.-B.; Clough, P. N.: Software Product Management and Pricing: Key Success Factors for Software Organizations. Springer, Berlin, Heidelberg, 2009.

[KMK12] Krebs, R.; Momm, C.; Kounev, S.: Architectural Concerns in Multi-Tenant SaaS Applications. In: Proceedings of the 2nd International Conference on Cloud Computing and Services Science (CLOSER). SciTePress, S. 426–431, 2012.

[LF14] Lewis, J.; Fowler, M.: Microservices: A Definition of this new Architectural Term, 2014, https://martinfowler.com/articles/microservices.html, Stand: 16. 10. 2026.

[MG11] Mell, P.; Grance, T.: The NIST Definition of Cloud Computing, NIST Special Publication 800-145, Gaithersburg: National Institute of Standards und Technology, 2011.

[Ne21] Newman, S.: Building Microservices: Designing Fine-Grained Systems. O'Reilly Media, Sebastopol, 2021.

[OP10] Osterwalder, A.; Pigneur, Y.: Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers. Wiley, Hoboken, 2010.

[OST16] Opara-Martins, J.; Sahandi, R.; Tian, F.: Critical Analysis of Vendor Lock-in and its Impact on Cloud Computing Migration: A Business Perspective. Journal of Cloud Computing 5 (4), 2016.

[Pe07] Peffers, K. et al.: A Design Science Research Methodology for Information Systems Research. Journal of Management Information Systems 24 (3), S. 45–77, 2007.

[PVC16] Parker, G. G.; Van Alstyne, M. W.; Choudary, S. P.: Platform Revolution: How Networked Markets Are Transforming the Economy and How to Make Them Work for You. W. W. Norton, New York, 2016.

[Ri11] Ries, E.: The Lean Startup. Crown Business, New York, 2011.

[RT03] Rochet, J.-C.; Tirole, J.: Platform Competition in Two-Sided Markets. Journal of the European Economic Association 1 (4), S. 990–1029, 2003.

[SA24] SAP SE: Describing Application Development, SAP Learning, 2024, https://learning.sap.com/courses/security-administrator/describing-application-development, Stand: 16. 10. 2026.

[SA26a] SAP SE: Account Model – SAP Business Technology Platform, 2026, https://help.sap.com/docs/btp/sap-business-technology-platform/account-model, Stand: 16. 10. 2026.

[SA26b] SAP SE: Build and Innovate with SAP – SAP PartnerEdge, Build, 2026, https://www.sap.com/partners/partner-program/build.html, Stand: 16. 10. 2026.

[SA26c] SAP SE: Capire – Dokumentation des SAP Cloud Application Programming Model, 2026, https://cap.cloud.sap/docs/, Stand: 16. 10. 2026.

[SA26d] SAP SE: Cloud Connector – SAP BTP Connectivity, 2026, https://help.sap.com/docs/connectivity/sap-btp-connectivity-cf/cloud-connector, Stand: 16. 10. 2026.

[SA26e] SAP SE: Exploring SAP BTP Commercial Models, SAP Learning, 2026, https://learning.sap.com/courses/becoming-an-sap-btp-solution-architect/exploring-sap-btp-commercial-models, Stand: 16. 10. 2026.

[SA26f] SAP SE: Extensibility – Capire, 2026, https://cap.cloud.sap/docs/guides/extensibility/, Stand: 16. 10. 2026.

[SA26g] SAP SE: Get Started with SAP BTP Enterprise Accounts, SAP Developer Center, 2026, https://developers.sap.com/tutorials/btp-ea-onboard-01-intro.html, Stand: 16. 10. 2026.

[SA26h] SAP SE: Multitenancy – Capire, 2026, https://cap.cloud.sap/docs/guides/multitenancy/, Stand: 16. 10. 2026.

[SA26i] SAP SE: Register a Multitenant Application to the SAP SaaS Provisioning Service, 2026, https://developers.sap.com/tutorials/register-multitenant-app-saas-provisioning-service.html, Stand: 16. 10. 2026.

[SA26j] SAP SE: SAP BTP Developer's Guide, 2026, https://help.sap.com/docs/btp/btp-developers-guide/what-is-btp-developers-guide, Stand: 16. 10. 2026.

[SA26k] SAP SE: SAP BTP Pricing, 2026, https://www.sap.com/products/technology-platform/pricing.html, Stand: 16. 10. 2026.

[SA26l] SAP SE: SAP Business Accelerator Hub, 2026, https://api.sap.com/, Stand: 16. 10. 2026.

[SA26m] SAP SE: SAP Business Technology Platform – Produktdokumentation, 2026, https://help.sap.com/docs/btp, Stand: 16. 10. 2026.

[SA26n] SAP SE: SAP Cloud SDK – Dokumentation, 2026, https://sap.github.io/cloud-sdk/, Stand: 16. 10. 2026.

[SA26o] SAP SE: SAP Discovery Center – Service Catalog and Estimator, 2026, https://discovery-center.cloud.sap/, Stand: 16. 10. 2026.

[SA26p] SAP SE: SAP Fiori Design Guidelines, 2026, https://www.sap.com/design-system/fiori-design-web/, Stand: 16. 10. 2026.

[SA26q] SAP SE: SAP Store, 2026, https://store.sap.com/, Stand: 16. 10. 2026.

[SF16] Simon, H.; Fassnacht, M.: Preismanagement: Strategie – Analyse – Entscheidung – Umsetzung. Springer Gabler, Wiesbaden, 2016.

[Sk13] Skok, D.: SaaS Metrics 2.0 – A Guide to Measuring and Improving What Matters, 2013, https://www.forentrepreneurs.com/saas-metrics-2/, Stand: 16. 10. 2026.

[SV99] Shapiro, C.; Varian, H. R.: Information Rules: A Strategic Guide to the Network Economy. Harvard Business School Press, Boston, 1999.

[Ti14] Tiwana, A.: Platform Ecosystems: Aligning Architecture, Governance, and Strategy. Morgan Kaufmann, Waltham, 2014.

A Wirtschaftlichkeitsmodell

A.1 Parameter des Basisszenarios

Tabelle A.1 enthält alle Annahmen des Basisszenarios. Sie sind im Wörterbuch `BASIS` des Skripts `wirtschaftlichkeit.py` im Verzeichnis `src` hinterlegt; Änderungen dort wirken sich nach einem erneuten Aufruf des Skripts auf alle Tabellen, Abbildungen und Zahlen im Text aus.

Tabelle A.1: Annahmen des Basisszenarios (Annahmen des Autors)

Parameter	Wert	Erläuterung
Investition (Jahr 0)		
Entwicklungsaufwand MVP	150 PT	Expertenschätzung, Tabelle 6.1
Interner Vollkostensatz	650 € je PT	Personal- und Gemeinkosten
Zertifizierung, Listing, Rechtsprüfung	12.000 €	einmalig
Plattformkosten Entwicklungsphase	4.800 €	DEV- und TEST-Subaccount
Markt und Erlöse		
Neukunden in Jahr 1 bis 5	5, 12, 18, 22, 25	Vertrieb über SAP Store und Partner
Kündigungsquote	10 % p. a.	bezogen auf den Bestand am Jahresanfang
Preise Paket S / M / L	490 / 990 / 1.890 €	je Mandant und Monat
Paketverteilung S / M / L	50 / 35 / 15 %	ergibt ARPA von 875 € je Monat
Laufende Kosten (Jahr 1 bis 5)		
Provision (Marktplatz, Partner)	15 % des Umsatzes	vertragsabhängig, pauschal angesetzt
Plattformkosten fix	22.800 € p. a.	HANA Cloud, Laufzeit, DEV/TEST
Plattformkosten variabel	300 € je Mandant p. a.	Container, Speicher, Protokolle
Support und Betrieb	0,6 PT je Mandant p. a.	inkl. Onboarding-Unterstützung
Wartung und Weiterentwicklung	35 PT p. a.	inkl. Anpassung an SAP-Releases
Marketing und Vertrieb fix	30.000 € p. a.	Listing, Inhalte, Veranstaltungen
Akquisitionskosten variabel	4.000 € je Neukunde	Vorvertrieb, Einrichtung
Partnerprogramm	3.000 € p. a.	Mitgliedsgebühr
Bewertung		
Kalkulationszins	10 %	risikoadjustiert, vor Steuern
Betrachtungszeitraum	Jahr 0 bis 5	ohne Restwert

A.2 Kernfunktion des Berechnungsmodells

Listing A.1 zeigt die Kernfunktion des Berechnungsmodells. Sie berechnet für einen Parametersatz die Jahreswerte von Kundenbestand, Umsatz, Auszahlungen und Cashflow sowie Kapitalwert, Amortisationsdauer und die Kennzahlen auf Kundenebene. Das vollständige Skript erzeugt zusätzlich die Szenarien, die Sensitivitätsanalyse und die Datendateien für die Abbildungen.

```python
def rechne(p):
    """Liefert eine Liste von Jahreswerten (Jahr 0 bis 5) und Kennzahlen."""
    arpa = arpa_monat(p) * 12
    i0 = p["pt_initial"] * p["tagessatz"] + p["partner_einmalig"] + p["plattform_entwicklung"]
    jahre = [{
        "jahr": 0, "kunden_ende": 0.0, "kunden_mittel": 0.0, "neu": 0,
        "umsatz": 0.0, "entwicklung": i0 - p["partner_einmalig"] - p["plattform_entwicklung"],
        "plattform": p["plattform_entwicklung"], "vertrieb": p["partner_einmalig"],
        "betrieb": 0.0,
    }]
    kunden = 0.0
    for t, neu in enumerate(p["neukunden"], start=1):
        start = kunden
        kunden = start * (1 - p["churn"]) + neu
        mittel = (start + kunden) / 2
        umsatz = mittel * arpa
        jahre.append({
            "jahr": t, "kunden_ende": kunden, "kunden_mittel": mittel, "neu": neu,
            "umsatz": umsatz,
            "entwicklung": p["wartung_pt"] * p["tagessatz"],
            "plattform": p["plattform_fix"] + mittel * p["plattform_var"],
            "vertrieb": umsatz * p["provision"] + p["marketing_fix"]
                        + neu * p["cac_var"] + p["partnerprogramm"],
            "betrieb": mittel * p["support_pt"] * p["tagessatz"],
        })
    kum = 0.0
    kapitalwert = 0.0
    amortisation = None
    for j in jahre:
        j["kosten"] = j["entwicklung"] + j["plattform"] + j["vertrieb"] + j["betrieb"]
        j["cashflow"] = j["umsatz"] - j["kosten"]
        vorher = kum
        kum += j["cashflow"]
        j["kumuliert"] = kum
        kapitalwert += j["cashflow"] / (1 + p["zins"]) ** j["jahr"]
        if amortisation is None and vorher < 0 <= kum:
            amortisation = j["jahr"] - 1 + (-vorher / j["cashflow"])
    # Unit Economics im eingeschwungenen Zustand (Jahr 5)
    j5 = jahre[-1]
    marge_kunde = arpa * (1 - p["provision"]) - p["plattform_var"] - p["support_pt"] * p["tagessatz"]
    cac = (p["marketing_fix"] + j5["neu"] * p["cac_var"]) / j5["neu"]
    ltv = marge_kunde / (p["zins"] + p["churn"])
    kz = {
        "arpa_monat": arpa / 12, "arpa": arpa, "investition": i0,
        "kapitalwert": kapitalwert, "amortisation": amortisation,
        "kumuliert5": kum, "kunden5": j5["kunden_ende"],
        "marge_kunde": marge_kunde, "marge_quote": marge_kunde / arpa,
        "cac": cac, "ltv": ltv, "ltv_cac": ltv / cac,
        "cac_payback_monate": cac / (marge_kunde / 12),
        "umsatz5": j5["umsatz"], "ebit_marge5": j5["cashflow"] / j5["umsatz"],
    }
    return jahre, kz
```

Listing A.1: Kernfunktion des Berechnungsmodells (`src/wirtschaftlichkeit.py`)